PROJECT REPORT

on

AI-Based Online Exam Proctoring System

Submitted by

Mr.Vikas Chaurasiya

Seat Number **510513626**

in partial fulfilment for the award of the degree

of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

under the guidance of

Dr. Sandeep Kamble

Department of Computer Science



Thakur Ramnarayan College of Arts and Commerce

(Semester V)

(2026 – 2027)

Thakur Ramnarayan College of Arts and Commerce

Thakur Ramnarayan Educational Campus, S.V. Road, Dahisar (East), Mumbai-400068

Department of Computer Science

CERTIFICATE

This is to certify that **Mr.Vikas Chaurasiya**, student of Thakur Ramnarayan College of Arts and Commerce Institute, studying in T.Y.B.Sc. (Computer Science), has successfully completed Mini Project1 titled **"AI-Based Online Exam Proctoring System"** in the prescribed area of specialization during the academic year 2026–2027. To the best of my knowledge, the work carried out by the student is original and the information presented in the project is true and correct.

Date of Submission:

Internal Guide

COORDINATOR/HOD
Department Computer Science

College Seal

Thakur Ramnarayan College of Arts and Commerce

Thakur Ramnarayan Educational Campus, S.V. Road, Dahisar (East), Mumbai-400068

DECLARATION

I, **Mr.Vikas Chaurasiya,** Student of Thakur Ramnarayan College of Arts and Commerce studying in T.Y.B.Sc. (C.S.), hereby declare that I have completed a project titled **"AI-Based Online Exam Proctoring System"** in the area of specialization for the academic year 2026-2027. I further declare that this work is original and that the information included in the project is true and correct to the best of my knowledge.

Name of the Student **Signature of Student**

ACKNOWLEDGEMENT

I express my sincere gratitude to our Principal, Head of the Department of Computer Science, and Project Guide **Dr. Sandeep Kamble** for their valuable guidance, academic insights, and continuous support throughout the development of this Software Requirements Specification.

I also extend my gratitude to the faculty members of the Computer Science Department at Thakur Ramnarayan College of Arts & Commerce for imparting the fundamental knowledge and technical resources required to complete this work successfully.

Finally, I express my heartfelt appreciation to my parents and friends for their steady motivation and constructive feedback throughout the course of this project.

Viikas Chaurasiya

Department of Computer Science TRCAC,

Mumbai

ABSTRACT

Online examinations have become an important method of conducting assessments in educational institutions. They provide flexibility, reduce the need for physical examination arrangements, and allow students to participate in tests through digital platforms. However, conducting examinations online also introduces several challenges related to examination management, time control, answer submission, result calculation, and monitoring student activity. In a traditional examination environment, invigilators supervise students to maintain discipline and reduce the possibility of unfair practices. In an online environment, maintaining similar supervision can be difficult because students may switch between browser tabs, navigate away from the examination page, or perform other activities during an ongoing test. These challenges create a requirement for an organized online examination platform that can manage examination activities and record selected browser-level events.

The AI-Based Online Exam Proctoring System is designed as a web-based examination management platform that aims to simplify the process of conducting multiple-choice question examinations. The system provides a centralized environment where administrators can create and manage examinations, while students can log in, view available examinations, attempt questions, submit answers, and access their results. Instead of depending on manual question distribution and evaluation, the platform manages the examination process through a browser-based interface connected to a backend application. The system is intended for academic use and focuses on providing a structured, reliable, and user-friendly examination experience.

One of the important capabilities of the system is user authentication and role-based access control. The application provides separate functionality for students and administrators. Students can access their examination dashboard and view examinations made available to them. Administrators are responsible for managing the question bank, configuring examinations, and reviewing examination records. Authentication ensures that users access the platform through valid credentials, while role-based authorization restricts access to functions according to the user's assigned role. This separation helps protect examination information and prevents students from accessing administrative operations or other students' records.

The system includes a Question Bank Management module through which administrators can create, update, and manage multiple-choice questions. Each question contains question text, multiple answer options, and a correct answer used for evaluation. Administrators can select questions while configuring an examination and organize them into the required examination structure. The system validates the information entered during question creation to reduce incomplete or incorrect records. Maintaining questions in a centralized database also makes examination preparation more organized and reduces the need to manually prepare question papers for every examination.

Another important feature is Examination Management. Administrators can configure an examination by specifying its title, duration, availability period, and selected questions. Students can view examinations that are available to them through their dashboard. When a student starts an examination, the backend verifies the examination's availability and creates an examination attempt. The system enforces the single-attempt rule for a student and examination according to the defined requirements. This helps maintain consistent examination records and prevents unauthorized repeated attempts.

The examination interface provides students with a question display, answer selection controls, and a visible countdown timer. Students can select their answers and submit them while the examination is active. The system stores selected answers against the corresponding examination attempt and question. A manual submission option is provided so that students can finish before the allotted time expires. The platform also supports automatic

submission when the examination deadline is reached. The backend remains responsible for validating the deadline, accepting or rejecting answer requests, and calculating the final score. This approach reduces dependence on the browser clock and helps maintain consistency in examination timing and evaluation.

The system also includes browser-level proctoring through the Page Visibility API. This browser feature allows the application to detect when the examination page becomes hidden, such as when a student switches to another browser tab or moves away from the examination window. When such an event occurs during an active examination, the client application sends the event information to the backend. The backend validates the examination attempt and records the event with a server-generated timestamp. Administrators can review these recorded events through the reporting functionality. This feature provides a basic monitoring mechanism and helps maintain an activity record during examinations. However, a visibility event alone does not prove that a student has engaged in unfair activity, because it may also occur due to notifications or other normal browser and operating-system actions.

From an architectural perspective, the system follows a web-based client-server approach. The frontend is developed using HTML5, CSS3, and JavaScript, which provide the interface and handle user interactions. The backend is developed using Python Flask, which manages application requests, authentication, examination operations, answer processing, and proctoring-event recording. SQLAlchemy is used as the Object Relational Mapper to interact with the relational database. SQLite can be used for demonstration and development purposes, while MySQL can be considered for a larger deployment. This architecture separates the user interface, application logic, and data storage, making the application easier to maintain and extend.

The database maintains important information such as user accounts, questions, examinations, examination attempts, submitted answers, and visibility-event records. Relationships between these records help maintain data consistency and support examination reporting. The backend uses stored answer keys to calculate scores after submission. Students can view their own examination results, while administrators can review examination attempts, scores, submission details, and recorded visibility events. This separation of information helps maintain the security and reliability of examination records.

Overall, the AI-Based Online Exam Proctoring System provides a structured platform for conducting and managing online MCQ examinations. By combining authentication, question management, examination scheduling, timed attempts, automatic submission, server-side evaluation, and browser visibility monitoring, the system aims to make online assessment more organized and manageable. Its modular web-based architecture provides a foundation for future improvements such as webcam-based monitoring, face verification, advanced activity analysis, enhanced reporting, and additional examination formats. These enhancements may be considered in future development, subject to technical feasibility and appropriate privacy requirements.

TABLE OF CONTENTS

No.	Chapter / Part	Sections
PART I – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)		
1	**INTRODUCTION**	1.1 Purpose
		1.2 Scope of the System
		1.3 Intended Audience
		1.4 Definitions, Acronyms, and Abbreviations
		1.5 Academic and Industry References
		1.6 Document Overview
2	**OVERALL SYSTEM DESCRIPTION**	2.1 Product Perspective & Architectural Paradigm
		2.2 Summary of Product Functions
		2.3 User Classes and Characteristics
		2.4 Operating & Execution Environment
		2.5 Design and Implementation Constraints
		2.6 System Assumptions and Dependencies
3	**DETAILED FUNCTIONAL REQUIREMENTS**	3.1 Module 1: Authentication & Authorization
		3.2 Module 2: Question Bank & Exam Configuration
		3.3 Module 3: Exam Attempt
		3.4 Module 4: Tab-Switch Detection & Proctoring
		3.5 Module 5: Result & Reporting
		3.6 Module 6: Administrative Dashboard
		3.7 Summary of Error Responses
4	**NON-FUNCTIONAL REQUIREMENTS**	4.1 Performance & Latency Benchmarks
		4.2 System Security & Integrity Standards
		4.3 Reliability & Availability
		4.4 Scalability & Capacity
		4.5 Usability & Human Factors
		4.6 Maintainability
5	**EXTERNAL INTERFACE REQUIREMENTS**	5.1 User Interface Requirements
		5.2 Hardware Interfaces
		5.3 Software Interfaces
		5.4 Communication Protocols
6	**DATABASE ARCHITECTURE & DATA DICTIONARY**	6.1 Relational Data Structure
		6.2 Entity Relationships
		6.3 Complete Data Dictionary

No.	Chapter / Part	Sections
		6.4 Data Retention
		6.5 Data Integrity
7	**SECURITY & ACCESS CONTROL MATRIX**	7.1 Role-Based Access Control (RBAC)
		7.2 Server-Side Security Controls
		7.3 Security Scope and Residual Risks
8	**COMPREHENSIVE TEST SUITE**	8.1 Verification Strategy
		8.2 Functional Test Cases
		8.3 Non-Functional Verification Plan
		8.4 Test Execution Record
9	**PROJECT TIMELINE & SDLC**	9.1 SDLC Methodology
		9.2 Milestone Schedule
10	**LIMITATIONS, FUTURE SCOPE & CONCLUSION**	10.1 System Limitations
		10.2 Future Scope
		10.3 Conclusion
PART II – UML DIAGRAM DOCUMENT		
1	**USE CASE DIAGRAM**	
2	**CLASS DIAGRAM**	
3	**SEQUENCE DIAGRAM – AUTHENTICATION**	
4	**SEQUENCE DIAGRAM – EXAM ATTEMPT**	
5	**SEQUENCE DIAGRAM – PROCTORING**	
6	**COMPONENT DIAGRAM**	
7	**ER DIAGRAM – DATABASE STRUCTURE**	
8	**ACTIVITY DIAGRAM – EXAM LIFECYCLE**	
	CONCLUSION	
PART III – ARCHITECTURE DESIGN DOCUMENT (ADD)		
1	**INTRODUCTION AND GOALS**	1.1 Purpose
		1.2 Project Overview
		1.3 Architectural Goals and Principle
2	**SYSTEM CONTEXT**	2.1 System Context Description
3	**OVERALL SYSTEM ARCHITECTURE**	3.1 Architecture Style
		3.2 Architecture Layers
4	**FRONTEND ARCHITECTURE**	4.1 Frontend Overview
		4.2 Frontend Responsibilities

No.	Chapter / Part	Sections
5	**BACKEND ARCHITECTURE**	5.1 Backend Overview
		5.2 Backend Components
6	**DATABASE ARCHITECTURE**	6.1 Database Overview
		6.2 Data Groups and Rules
7	**AUTHENTICATION AND AUTHORIZATION ARCHITECTURE**	7.1 Authentication and Authorization Overview
		7.2 Control Points
8	**DATA FLOW PATTERNS**	8.1 Common Request Path
		8.2 Main Data Flows
9	**SECURITY ARCHITECTURE**	9.1 Security Overview
		9.2 Security Controls
		9.3 Security Responsibilities
10	**DEPLOYMENT ARCHITECTURE**	10.1 Deployment Overview
		10.2 Deployment Components
11	**KEY DESIGN DECISIONS**	
12	**FUNCTIONAL ARCHITECTURE**	12.1 Student Functional Architecture
		12.2 Administrator Functional Architecture
13	**NON-FUNCTIONAL ARCHITECTURE**	
14	**TECHNOLOGY STACK**	
15	**REQUIREMENT-TO-ARCHITECTURE TRACEABILITY**	
16	**FUTURE ARCHITECTURAL ENHANCEMENTS**	
17	**ARCHITECTURAL SUMMARY**	
18	**CONCLUSION**	
PART IV – SCREENSHOTS, APPENDICES & REFERENCES		
1	**USER INTERFACE SCREENSHOTS – STUDENT**	1.1 Login Screen
		1.2 Student Dashboard
		1.3 Exam Interface
		1.4 Manual Submission Confirmation
		1.5 Result Screen
2	**OUTPUT OF THE SYSTEM – ADMINISTRATOR CONSOLE**	2.1 Admin Dashboard
		2.2 Question Bank Manager
		2.3 Exam Configuration
		2.4 Consolidated Report Viewer
		2.5 CSV Export Output

No.	Chapter / Part	Sections
		2.6 Student Account Overview
3	**SUPPORTING ATTACHMENTS**	
	APPENDICES	A. REST Endpoint Summary; B. Requirement Identifier Index; C. Implemented, Specified and Proposed Functionality
	REFERENCES	

1. INTRODUCTION

1.1 Purpose

The primary purpose of this **Software Requirements Specification (SRS)** is to describe, clearly and in a structured form, the functional and non-functional requirements of the **AI-Based Online Exam Proctoring System**, Version 1.0. The system is a web application that allows an institution to conduct timed, multiple-choice examinations online while automatically logging tab-switching behaviour as a lightweight proctoring signal. The SRS defines what the system shall do, who uses it, what information it stores and processes, and which constraints it must respect.

The document specifies **what the system shall do rather than how each component is implemented**. Implementation detail appears only where it constrains behaviour. For example, the way the Page Visibility API works determines which event data the system is able to capture, and the auto-submit requirement dictates how the exam timer must be synchronised between the client and the server. Keeping the specification at this level makes it a stable reference even if the internal code structure changes.

For the **student developer**, the SRS is the engineering reference for the browser interface, the Flask REST endpoints, the SQLAlchemy data model and the validation rules. It ensures that every implemented feature corresponds to a documented requirement and that the build stays within the six-week academic timeline. For the **project guide and any evaluator**, it provides a formal basis for judging whether the project meets the expected academic and technical standard, because the intended functionality, boundaries and limitations are written down before they are examined.

The document is also valuable for **testing**. Each functional requirement is stated with its input, process, output and error cases, so that it can be converted directly into a test case. Non-functional requirements carry measurable targets, such as response times and timer accuracy, which can likewise be verified. Finally, the SRS gives **future maintainers** a record of the original design intent, including the reasons behind decisions such as hiding the violation count from students and timestamping violations on the server.

1.2 Scope of the System

The system is a **browser-based examination platform built around two roles: Student and Admin**. A student logs in, attempts a timed multiple-choice exam drawn from a question bank, and is automatically submitted when the timer expires. While the exam is in progress, the client detects every instance of the student navigating away from the exam tab or window and reports it to the server, which compiles the events into a per-student violation log. The admin reviews a consolidated report that combines each student's score with their tab-switch history.

The capabilities listed in the table below are in scope for the initial academic build.

Capability	Description
Role-Based Login	Separate authenticated sessions for students and the admin, each routed to a distinct dashboard.
Timed MCQ Exam	A fixed-duration multiple-choice test drawn from a question bank stored in the database.

Capability	Description
Auto-Submit	The exam is submitted automatically, with whatever has been answered so far, the instant the countdown reaches zero.
Tab-Switch Detection	A `visibilitychange` listener on the exam page detects and timestamps every departure from the exam tab.
Result & Proctoring Report	A combined view, per student, of the score obtained and the violation log recorded during the attempt.

Out of scope for this build: webcam or audio-based proctoring, face detection, browser lock-down or fullscreen enforcement, plagiarism or answer-similarity detection, and PDF export of reports. These items are recorded as future scope in Section 10.2 of this report.

It is important to read the scope correctly. The system captures **one specific, well-defined behaviour**, leaving the exam tab, and presents it to a human reviewer next to the score. It does not attempt to replicate a human invigilator, it does not verify the identity of the person taking the exam, and it does not claim to detect or prevent every form of malpractice. The word "AI-Based" in the project title reflects the direction of the project (the stated future scope includes OpenCV-based face detection); the build specified in this report is limited to browser-level tab-switch proctoring.

1.3 Intended Audience

This SRS is written for the groups involved in developing, evaluating, testing and later extending the system. Each group can use it for a different purpose while sharing one common understanding of the requirements.

- **Project Guide and Evaluators:** use the SRS to understand the purpose, scope, technical approach and limitations of the system and to judge it against the expectations of a college minor project.
- **Student Developer:** uses it as the engineering reference during implementation of the browser interface, REST endpoints, database model and validation logic.
- **Testers and Quality Reviewers:** convert the stated inputs, processes, outputs and error cases into test cases (see Chapter 8) and verify the measurable non-functional targets.
- **Database Maintainers:** use the data requirements in Chapter 6 to understand entities, relationships and integrity constraints.
- **Future Maintainers and Extenders:** use it to understand the original design intent before adding features such as face detection or PDF export.

1.4 Definitions, Acronyms, and Abbreviations

The terms below are used throughout this report with the meanings given here.

Term / Acronym	Full Definition
API	Application Programming Interface: the REST endpoints exposed by the backend to the browser client
Auto-Submit	Submission of an exam attempt when its allotted time elapses, regardless of how many questions have been answered

Term / Acronym	Full Definition
Beacon API	Browser interface (`navigator.sendBeacon`) used to send a small request reliably while a page is closing
bcrypt	A salted password-hashing algorithm named in the SRS as an example of a suitable hashing method
CSV	Comma-Separated Values: the export format for the consolidated admin report
ER Diagram	Entity-Relationship Diagram showing database entities, key fields and cardinalities
Exam Attempt	A single instance of a student taking a scheduled exam, from start to submission
Flask	Lightweight Python web framework used for the REST backend
Gunicorn	A WSGI HTTP server named as an example for running Flask beyond local development
HTTP / HTTPS	Hypertext Transfer Protocol / its secure (TLS-encrypted) variant
JSON	JavaScript Object Notation: the data format for all REST payloads
MCQ	Multiple-Choice Question: a question with a fixed set of options, exactly one of which is correct
ORM	Object-Relational Mapping: here, SQLAlchemy
Page Visibility API	Browser API exposing the `visibilitychange` event and `document.hidden` property
Question Bank	The stored collection of MCQ items from which an exam is composed
RBAC	Role-Based Access Control
REST	Representational State Transfer: the style of the backend API
Session Token	Server-issued credential proving an authenticated login, used to authorise subsequent requests
SQLAlchemy	Python ORM used for all database access
SRS	Software Requirements Specification
Tab-Switch / Visibility Event	A browser-detected transition of the exam tab between visible and hidden states
UML	Unified Modeling Language
Violation	A logged tab-switch event associated with a specific exam attempt, carrying a server-assigned timestamp
Violation Log	The ordered set of violations recorded against one exam attempt
WSGI	Web Server Gateway Interface: the standard between Python web applications and web servers

1.5 Academic and Industry References

1. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
2. Project Proposal: AI-Based Online Exam Proctoring System, College Minor Project Proposal, submitted by Vikas.
3. MDN Web Docs, Page Visibility API (`visibilitychange` event).
4. Flask official documentation; SQLAlchemy ORM documentation.
5. OWASP Top 10 Web Application Security Risks.

1.6 Document Overview

This report is organised in four parts. **Part I (Chapters 1 to 10)** is the Software Requirements Specification: introduction, overall description, detailed functional requirements, non-functional requirements, external interfaces, database architecture and data dictionary, security and access control, test plan, project timeline and SDLC, and limitations, future scope and conclusion.

Part II, the UML Diagram Document, presents eight simplified diagrams that translate the SRS into structural, behavioural, architectural and data views: the use-case diagram, class diagram, three sequence diagrams (authentication, exam attempt, proctoring), the component diagram, the entity-relationship diagram and the exam lifecycle activity diagram. **Part III, the Architecture Design Document,** converts the SRS into a technical architecture covering responsibilities, interfaces, data ownership, security controls and deployment boundaries. **Part IV** contains the screenshot and output pages, appendices and the references list.

Diagrams and screenshots appear in this report as clearly marked placeholders. They must be replaced with the final diagrams and with real screenshots of the running system before submission.

2. OVERALL SYSTEM DESCRIPTION

2.1 Product Perspective & Architectural Paradigm

The system is a **self-contained academic project** rather than a component of a larger institutional suite. It addresses a specific, narrow gap: existing free quiz tools (Google Forms, generic MCQ plug-ins) capture answers but capture no signal about whether the student stayed on the exam page throughout the attempt. A physically invigilated exam has a human watching for exactly this; an unmonitored online exam has nothing in its place.

The system supplies that missing signal at the **browser level**. It does not attempt to replicate a human invigilator or verify identity. It captures one specific behaviour (leaving the exam tab) and surfaces it to the admin alongside the score, so that a suspiciously high number of tab switches can be weighed by a human reviewer rather than going unrecorded.

Architecturally the system follows a simple **three-layer model**. The client is a browser-based interface (login, exam interface, countdown timer and result view). The application backend is a Python Flask REST service that handles authentication, exam delivery, answer capture, scoring and violation logging. The database is SQLite for local development and testing, with a migration path to MySQL for a shared or hosted deployment. Because the exam timer and the violation log are both integrity-sensitive, the backend is the authority for elapsed time and for the recorded event history: the client reports what it observes, but the server independently timestamps every request and decides when an attempt has expired and what the final violation count is.

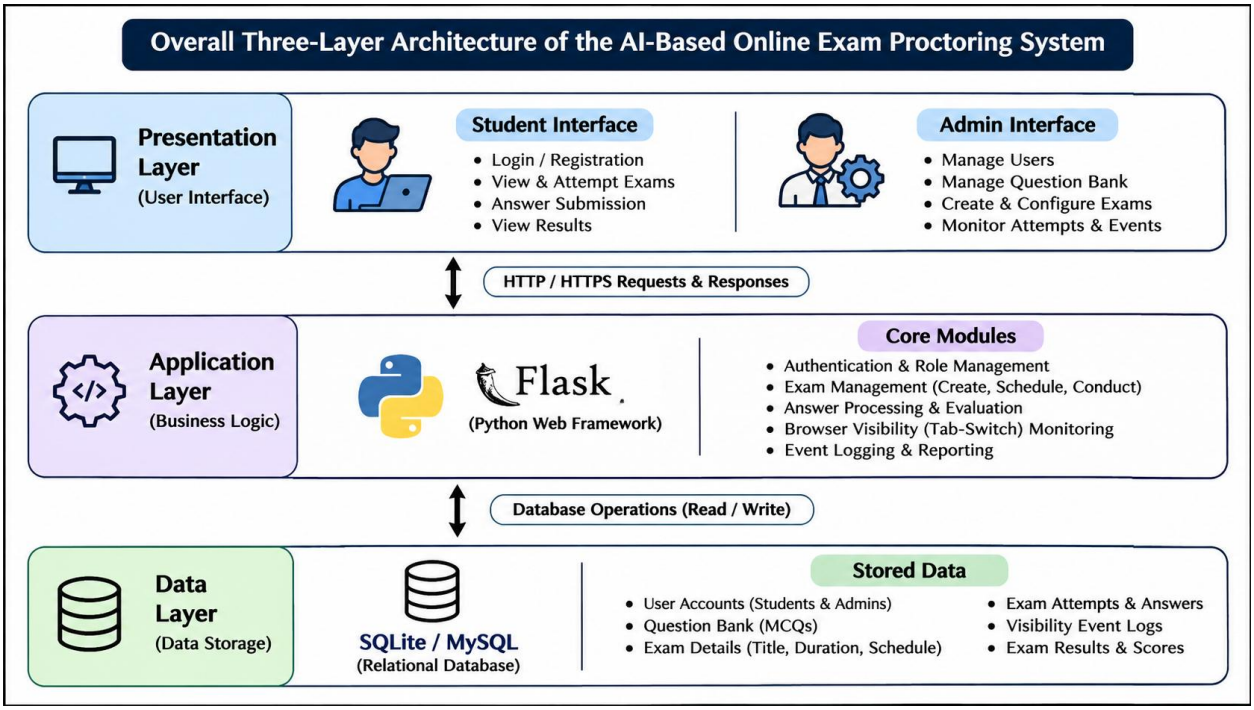


Figure S.1: Overall three-layer architecture of the AI-Based Online Exam Proctoring System

The system integrates with the following external elements:

Integration	Role in the System
Relational Database (SQLite / MySQL)	Stores the question bank, student accounts, exam attempts, answers and violation logs.

Integration	Role in the System
Browser Page Visibility API	The client-side source of tab-switch events (`visibilitychange`, `document.hidden`).
Git / GitHub	Version control and public hosting of the project for reproducibility, per the project's own objectives.

2.2 Summary of Product Functions

At a high level the system provides the following functions. Each is specified in detail in Chapter 3.

- **Account Access:** role-based login for students and the admin, with logout and idle-session expiry.
- **Question Bank Management:** admin creation, editing and removal of MCQ items, each with four options and exactly one correct option.
- **Exam Delivery:** presentation of a timed set of questions to a logged-in student within the exam's availability window.
- **Answer Capture:** recording of the student's selected option per question as the exam progresses; a changed selection overwrites the earlier one.
- **Countdown & Auto-Submit:** a visible timer, reconciled with the server deadline, that forces submission when it reaches zero.
- **Tab-Switch Detection:** client-side capture of every visibility change during an active attempt.
- **Violation Logging:** server-side recording of each detected event with a server-assigned timestamp.
- **Scoring:** automatic comparison of submitted answers against the answer key.
- **Result Display:** the student's own score shown after submission.
- **Admin Reporting:** a consolidated per-student view of score plus violation history, with CSV export.

2.3 User Classes and Characteristics

The system has two user classes. They differ in technical proficiency, in the data they may access, and in what they are allowed to do.

User Role	Technical Literacy	Key Operational Goals	Permissions Level
Student	Basic; familiar with web browsers and online forms	Log in, start an assigned exam, answer MCQs within the time limit, view own score after submission.	Own attempts, answers and results only. No access to the question bank, other students' data or the admin dashboard.
Admin	Moderate; operates a dedicated administrative section	Maintain the question bank, configure exams, review the consolidated result and proctoring report after an exam window closes.	Full access to the question bank, exam configuration and every student's results and violation logs. No exam-taking role.

Student

- **Authentication:** username or email and password; a session token is issued on successful login.
- **Typical tasks:** log in, start an assigned exam, answer MCQs within the time limit, view their own score after submission.
- **Constraints:** cannot view the answer key, cannot re-attempt a submitted exam, and cannot see their own violation count during the attempt. The count is visible only in the admin's post-exam report, so that awareness of the count cannot be used to game the detector.

Admin

- **Authentication:** the same login mechanism as a student, distinguished by a role flag on the account.
- **Typical tasks:** create and edit MCQ questions, configure exam duration and question set, and review the consolidated result and proctoring report after an exam window closes.
- **Constraints:** cannot edit a question that belongs to an exam for which attempts are already in progress, to avoid changing the basis on which those attempts are being scored.

2.4 Operating & Execution Environment

Environment Component	Requirement Specification
Client Side	Any evergreen desktop browser (Chrome, Firefox, Edge) supporting the Page Visibility API. The exam interface is not designed for mobile browsers, because tab-switch semantics differ on mobile.
Client Technology	HTML5, CSS3, vanilla JavaScript (ES6+)
Server Side	Python 3.10+, Flask, served behind a WSGI server (for example Gunicorn) for anything beyond local development
Database	SQLite for development and demonstration; MySQL as the production-equivalent option, accessed through SQLAlchemy
Network	A stable internet connection for the duration of the exam; plain HTTP on loopback for local development, HTTPS otherwise
Deployment	A single backend instance is sufficient at project scale; source hosted on GitHub with a documented local setup procedure

2.5 Design and Implementation Constraints

The following constraints bind the design and implementation. Most of them exist to protect the integrity of the exam record.

- **Server-authoritative time:** the server is the sole authority for elapsed exam time. The client-side timer is a display convenience, not the enforcement mechanism, so a request arriving after the server-computed deadline is rejected.
- **Server-side scoring:** every scored action (answer submission, exam finalisation) is validated on the server; the client never determines or submits a score.
- **Password handling:** passwords are never stored or logged in plain text.

- **Server-side violation timestamps:** a tab-switch event is timestamped by the server on receipt, not trusted from a client-supplied timestamp alone, so that client clock drift cannot distort the violation log.
- **Single active attempt:** a student may hold at most one active attempt per exam; a second concurrent attempt from another tab or device is rejected.
- **Stable answer key:** once an exam is published, its questions are not altered in a way that would change the answer key for attempts already in progress.
- **Academic timeline:** the build is scoped to a six-week development timeline, which bounds the feature set to the requirements listed in Chapter 3.

2.6 System Assumptions and Dependencies

- Students have access to a desktop or laptop with a modern browser and a stable internet connection for the duration of the exam.
- The institution provisions student accounts (or approves registrations) and the single admin account out of band or through an admin-managed onboarding step.
- Tab-switch detection is a deterrent and reporting signal, not a guarantee. A student can defeat it with a second physical device, a virtual machine, or by disabling JavaScript. The system does not claim to prevent malpractice, only to record one specific class of it.
- The academic build runs against a local or lab-hosted database and is not hardened for public internet exposure. Any deployment outside the classroom or lab context would need additional security review.

3. DETAILED FUNCTIONAL REQUIREMENTS

This chapter specifies the functional requirements of the system, grouped into six modules. Every requirement has a unique identifier of the form `FR-<MODULE>-<NN>` and, where applicable, is described by its input, process, output and error cases. HTTP status codes are given where the SRS fixes the expected error behaviour. The requirement identifiers are reused in the traceability matrix (Part III) and in the test plan (Chapter 8).

The table below is an index of all requirements in this chapter.

Requirement ID	Requirement Title	Module
FR-AUTH-01	Student Registration / Provisioning	Module 1
FR-AUTH-02	Login	Module 1
FR-AUTH-03	Logout / Session Expiry	Module 1
FR-AUTH-04	Role-Based Authorization	Module 1
FR-QB-01	Question Creation	Module 2
FR-QB-02	Question Edit & Removal	Module 2
FR-QB-03	Exam Configuration	Module 2
FR-QB-04	Exam Listing	Module 2
FR-EXAM-01	Exam Start	Module 3
FR-EXAM-02	Answer Capture	Module 3
FR-EXAM-03	Countdown Timer	Module 3
FR-EXAM-04	Auto-Submit on Timer Expiry	Module 3
FR-EXAM-05	Manual Submission	Module 3
FR-EXAM-06	Single Active Attempt Enforcement	Module 3
FR-PROC-01	Visibility Change Detection	Module 4
FR-PROC-02	Violation Reporting	Module 4
FR-PROC-03	Server-Side Violation Logging	Module 4
FR-PROC-04	Violation Count Concealment During Attempt	Module 4
FR-RES-01	Automatic Scoring	Module 5
FR-RES-02	Student Result View	Module 5
FR-RES-03	Consolidated Admin Report	Module 5
FR-RES-04	Result Export	Module 5
FR-ADM-01	Admin Dashboard	Module 6
FR-ADM-02	Student Account Overview	Module 6

3.1 Module 1: Authentication & Authorization

FR-AUTH-01: Student Registration / Provisioning

- **Description:** The system shall allow a new student account to be created, either through self-registration or admin provisioning.
- **Input:** Full name, email or username, password (minimum 8 characters, mixed case and numeric).
- **Process:** Validate uniqueness of the email or username; hash the password using a salted hashing algorithm (for example bcrypt); create the account in an active state.
- **Output:** Registration acknowledgement.
- **Error Cases:** Duplicate email or username: HTTP 409. Password below policy: HTTP 422.

FR-AUTH-02: Login

- **Description:** The system shall authenticate a registered user and issue a session credential.
- **Input:** Email or username and password.
- **Process:** Look up the account; verify the password against the stored hash; on success, issue a session token and identify the account's role.
- **Output:** Session token and role (student or admin), used to route to the correct dashboard.
- **Error Cases:** Invalid credentials: HTTP 401. Disabled account: HTTP 403.

FR-AUTH-03: Logout / Session Expiry

- **Description:** The system shall allow a user to end their session and shall expire sessions after a period of inactivity.
- **Process:** On logout, invalidate the session token server-side. Idle sessions expire automatically after a configured timeout.

FR-AUTH-04: Role-Based Authorization

- **Description:** The system shall enforce role checks on every protected endpoint.
- **Process:** A server-side check on each request confirms that the session is valid and that the account's role is permitted for the requested action before the handler executes.
- **Error Cases:** Missing or invalid session: HTTP 401. Valid session without the required role: HTTP 403.

3.2 Module 2: Question Bank & Exam Configuration

FR-QB-01: Question Creation

- **Description:** The admin shall be able to add a new MCQ item to the question bank.
- **Input:** Question text, four answer options, the index of the correct option, and an optional topic or category tag.
- **Process:** Validate that exactly one option is marked correct and that all option fields are non-empty; store the item in the question bank.
- **Output:** Question identifier.
- **Error Cases:** No option marked correct, or more than one: HTTP 422.

FR-QB-02: Question Edit & Removal

- **Description:** The admin shall be able to edit or remove an existing question.
- **Process:** Editing or removing a question already attached to a published exam with attempts in progress is blocked, so that the basis for scoring cannot shift mid-exam.

- **Error Cases:** Edit attempted on a question locked by an in-progress exam: HTTP 409.

FR-QB-03: Exam Configuration

- **Description:** The admin shall be able to define an exam by selecting questions from the bank and setting its parameters.
- **Input:** Exam title, duration in minutes, the set of question identifiers to include, and a scheduled availability window.
- **Process:** Validate that the question set is non-empty and that the duration is a positive value; store the exam definition.
- **Output:** Exam identifier and a summary for confirmation.
- **Error Cases:** Empty question set: HTTP 422. Duration outside the permitted 5 to 180 minute range: HTTP 422.

FR-QB-04: Exam Listing

- **Description:** The system shall list available exams to a student and all defined exams to the admin, filtered appropriately by role and availability window.

3.3 Module 3: Exam Attempt

FR-EXAM-01: Exam Start

- **Description:** The system shall allow a student to begin a single attempt on an exam that is within its availability window.
- **Process:** Verify that the student has no existing attempt on this exam; create an attempt record with a server-recorded start time; serve the configured question set without the correct-answer indicator.
- **Output:** Attempt identifier, question set, and server-computed remaining time.
- **Error Cases:** Exam outside its availability window: HTTP 403. Attempt already exists: HTTP 409.

FR-EXAM-02: Answer Capture

- **Description:** The system shall record a student's selected option for a question as the exam progresses.
- **Input:** Attempt identifier, question identifier, selected option.
- **Process:** Verify that the attempt belongs to the requesting student and is still within its time limit before accepting the answer; upsert the answer against the attempt so that a changed selection overwrites the prior one.
- **Error Cases:** Attempt already submitted or expired: HTTP 409.

FR-EXAM-03: Countdown Timer

- **Description:** The client shall display a countdown of the time remaining in the attempt, synchronised against the server-computed deadline.
- **Process:** On exam start and periodically thereafter, the client reconciles its displayed timer against the server's remaining-time value, so that client-side clock drift does not desynchronise the displayed countdown from the actual deadline.

FR-EXAM-04: Auto-Submit on Timer Expiry

- **Description:** The system shall submit the attempt automatically, using whatever answers have been recorded, the instant the allotted time elapses.
- **Process:** The client triggers a submission request when its countdown reaches zero.

Independently, the server rejects any further answer-capture request received after the attempt's computed deadline and treats the attempt as due for submission regardless of whether the client's auto-submit request arrives.
- **Output:** Attempt marked submitted, with its submission reason recorded as either manual or auto.

FR-EXAM-05: Manual Submission
- **Description:** The system shall allow a student to submit the attempt voluntarily before time expires.
- **Process:** Mark the attempt submitted at the current server time; no further answers are accepted for this attempt.
- **Error Cases:** Attempt already submitted: HTTP 409.

FR-EXAM-06: Single Active Attempt Enforcement
- **Description:** The system shall prevent a student from holding more than one concurrent attempt on the same exam, including from a second browser tab or device.
- **Process:** The attempt-start check (FR-EXAM-01) and every answer-capture and submission request are all scoped to a single attempt record per student per exam.

3.4 Module 4: Tab-Switch Detection & Proctoring

FR-PROC-01: Visibility Change Detection
- **Description:** The client shall detect every transition of the exam tab between visible and hidden states during an active attempt.
- **Process:** A `visibilitychange` event listener is attached to the exam page for the duration of the attempt; each transition to hidden is captured as a candidate violation.
- **Rationale:** The Page Visibility API is a well-supported, low-overhead browser signal for exactly this transition, matching the project's intent of lightweight, JavaScript-only proctoring without requiring camera or microphone access.

FR-PROC-02: Violation Reporting
- **Description:** The client shall report each detected tab-switch event to the server for the active attempt.
- **Input:** Attempt identifier, client-observed event type (hidden or visible).
- **Process:** The event is sent to the server as soon as it is detected. Where the tab is being closed, the report uses a delivery method that does not depend on the page remaining open (for example the Beacon API), so that a closing tab does not silently drop the last event.

FR-PROC-03: Server-Side Violation Logging
- **Description:** The system shall record each reported tab-switch event against the attempt with a server-assigned timestamp.
- **Process:** On receipt of a violation report, verify that the attempt is active, then append an entry to the violation log carrying the server's own timestamp rather than any client-supplied time.
- **Output:** Violation recorded; running violation count updated for the attempt.
- **Rationale:** Timestamping on the server, independent of the client, prevents a manipulated client clock from producing a misleading violation log.

FR-PROC-04: Violation Count Concealment During Attempt

- **Description:** The system shall not reveal a student's own running violation count to them during the attempt.
- **Process:** The exam interface never requests or displays the violation count; it is retrievable only through the admin report (FR-RES-03).
- **Rationale:** Surfacing the count during the exam would let a student calibrate exactly how much tab-switching is "safe", defeating the purpose of the detector.

3.5 Module 5: Result & Reporting

FR-RES-01: Automatic Scoring

- **Description:** The system shall score a submitted attempt automatically against the answer key.
- **Process:** On submission (manual or auto), compare each recorded answer to the correct option for its question and compute the total score.
- **Output:** Score recorded against the attempt.

FR-RES-02: Student Result View

- **Description:** The system shall show a student their own score once their attempt is submitted and scored.
- **Output:** Score, total questions and submission time; no visibility of the violation log or of other students' results.

FR-RES-03: Consolidated Admin Report

- **Description:** The system shall present the admin with a per-student view combining score and violation history for a given exam.
- **Output:** For each student who attempted the exam: score, submission reason (manual or auto), total violation count, and the full timestamped violation log.

FR-RES-04: Result Export

- **Description:** The system shall allow the admin to export the consolidated report as a CSV file for record-keeping.
- **Output:** A downloadable CSV listing one row per student attempt with score and violation count.

3.6 Module 6: Administrative Dashboard

FR-ADM-01: Admin Dashboard

- **Description:** The admin home screen shall summarise platform activity.
- **Output:** Number of exams configured, number of active attempts in progress, and number of submitted attempts awaiting review.

FR-ADM-02: Student Account Overview

- **Description:** The admin shall be able to view the list of registered students and their exam history.

3.7 Summary of Error Responses

The SRS fixes the following HTTP error behaviour across the modules.

HTTP Status	Meaning in this System	Raised By
401	Invalid credentials, or a missing or invalid session token	FR-AUTH-02, FR-AUTH-04
403	Disabled account; valid session without the required role; exam outside its availability window	FR-AUTH-02, FR-AUTH-04, FR-EXAM-01
409	Duplicate email or username; edit of a question locked by an in-progress exam; attempt already exists; attempt already submitted or expired	FR-AUTH-01, FR-QB-02, FR-EXAM-01, FR-EXAM-02, FR-EXAM-05
422	Password below policy; no option or more than one option marked correct; empty question set; duration outside 5 to 180 minutes	FR-AUTH-01, FR-QB-01, FR-QB-03

4. NON-FUNCTIONAL REQUIREMENTS

The non-functional requirements describe how well the system must perform its functions. Targets are stated for lab or classroom scale, which is the intended use of the academic build.

4.1 Performance & Latency Benchmarks

Metric	Requirement
API response time	Under 500 ms for read endpoints and under 1 s for answer-capture and submission endpoints, under normal lab-scale load.
Exam interface load	Interactive within 2 seconds on a standard broadband connection.
Timer accuracy	Displayed countdown stays within 2 seconds of the server-computed deadline over a full exam duration.
Violation report delivery	A tab-switch event is delivered to the server within 1 second of the client detecting it, network conditions permitting.
Concurrent users	At least 50 concurrent exam attempts at classroom or lab scale.

4.2 System Security & Integrity Standards

Requirement	Detail
Authentication	Session-token-based login; tokens are invalidated on logout and expire after a period of inactivity.
Password storage	Passwords are hashed with a salted algorithm; plaintext passwords are never logged or persisted.
Transport security	HTTPS is used for any deployment beyond local development; local development may use plain HTTP on loopback only.
Authorization	Role and ownership checks are enforced server-side on every protected endpoint (see FR-AUTH-04).
Input validation	All request payloads are validated server-side; parameterised queries and ORM binding prevent SQL injection.
Answer key protection	The correct-answer indicator is never sent to the client while an exam is in progress.
Violation log integrity	Event timestamps are assigned server-side (see FR-PROC-03), not trusted from the client.

4.3 Reliability & Availability

- An exam attempt's timer state is derived from the server-recorded start time on every request, so a page refresh or brief network drop does not reset or extend the student's remaining time.
- Auto-submission at the deadline is enforced server-side (FR-EXAM-04) independently of whether the client's own auto-submit request successfully arrives, so a closed or frozen tab does not leave an attempt unresolved.
- A violation report sent via the Beacon API on tab or window close is **best-effort**, per the

browser's own guarantees. The system does not claim delivery is guaranteed in every case, only that it is attempted without depending on the page remaining open.

4.4 Scalability & Capacity

- The backend is structured so that the API layer can be run behind a WSGI server capable of multiple worker processes if concurrent load grows beyond a single classroom.
- Database indexes are defined on the attempt-lookup columns (student, exam) that back the single-active-attempt check (FR-EXAM-06) and the report query (FR-RES-03).
- The data layer is accessed through SQLAlchemy, which keeps the path from SQLite to MySQL open for shared deployments.

4.5 Usability & Human Factors

- The exam interface displays the countdown timer prominently at all times during an attempt.
- A confirmation prompt is shown before manual submission, stating the number of questions left unanswered, if any.
- Failure and validation states are reported in plain language, stating what happened and what the user may do next.
- The admin's question-creation form validates the answer-key selection inline, before the question can be saved.

4.6 Maintainability

- Clear separation between route handlers, business logic and data-access code, so that scoring and violation-logging logic is not duplicated across endpoints.
- Environment-specific configuration (database URL, secret key) is supplied through environment variables, not hard-coded.
- The project is version-controlled in Git and hosted on GitHub with setup instructions, per the project's own reproducibility objective.
- Core logic (scoring, timer-expiry enforcement and violation logging) is to be covered by automated tests.

5. EXTERNAL INTERFACE REQUIREMENTS

5.1 User Interface Requirements

The user interface consists of the following screens. Screenshots of each screen are to be inserted in Part IV once the interface is available.

Screen	Role	Description
Login	Public	Username or email and password entry, routed to the appropriate dashboard on success.
Student Dashboard	Student	List of assigned exams within their availability window and past results.
Exam Interface	Student	MCQ presentation, answer selection, visible countdown timer, and manual submit control.
Result Screen	Student	Score and submission summary shown after an attempt is scored.
Admin Dashboard	Admin	Platform activity summary: exam count, attempts in progress, attempts awaiting review.
Question Bank Manager	Admin	Create, edit and remove MCQ items.
Exam Configuration	Admin	Define exam title, duration, question set and availability window.
Report Viewer	Admin	Consolidated per-student score and violation log, with CSV export.

5.2 Hardware Interfaces

- Standard keyboard and mouse or trackpad input on a desktop or laptop.
- No camera, microphone or specialised peripheral is required for this build.
- Local or session storage in the browser is used only for the client-side session token, not for exam content or answers.

5.3 Software Interfaces

System	Interface
Database (SQLite / MySQL)	SQLAlchemy ORM over the database's native connection protocol.
Browser Page Visibility API	The `document.visibilitychange` event and `document.hidden` property, consumed client-side in JavaScript.
Browser Beacon API	`navigator.sendBeacon`, used to deliver a final violation report reliably on tab or window close.

System	Interface
Git / GitHub	Source control and public repository hosting for project submission and reproducibility.

5.4 Communication Protocols

- **Protocol:** HTTPS for any non-local deployment; local development may use plain HTTP on loopback.
- **Data format:** JSON for all REST payloads.
- **Authentication header:** a session token supplied on each authenticated request (for example as a bearer token or secure cookie).
- **Call pattern:** the exam-attempt, answer-capture and violation-report endpoints are the only ones invoked repeatedly during an active exam session; all other endpoints are invoked on demand.

6. DATABASE ARCHITECTURE & DATA DICTIONARY

6.1 Relational Data Structure

The system stores its data in a relational database: SQLite for development and demonstration, MySQL as the production-equivalent option, both accessed through SQLAlchemy. The data model contains seven entities. Questions persist independently of any one exam, and attempts, answers and violation logs remain available for review and project evaluation.

Entity	Table	Key Fields
User	users	id, full_name, email, password_hash, role, created_at
Question	questions	id, text, option_a, option_b, option_c, option_d, correct_option, topic, created_by
Exam	exams	id, title, duration_minutes, available_from, available_to, created_by
Exam Question	exam_questions	id, exam_id, question_id
Exam Attempt	exam_attempts	id, exam_id, student_id, started_at, submitted_at, submission_reason, score
Answer	answers	id, attempt_id, question_id, selected_option, answered_at
Violation Log Entry	violations	id, attempt_id, event_type, occurred_at (server-assigned)

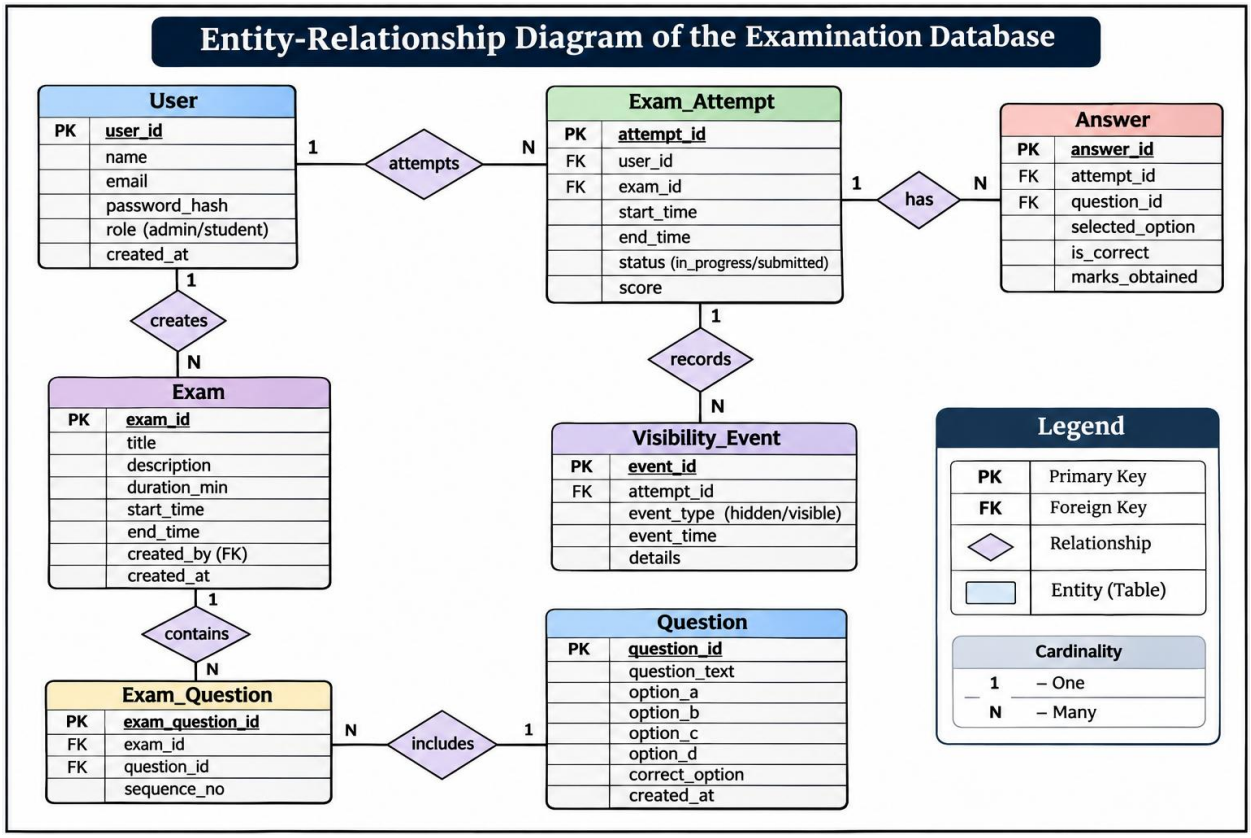


Figure S.2: Entity-Relationship diagram of the examination database

6.2 Entity Relationships

- A User with role Admin creates many Questions and Exams; a Question or Exam is created by exactly one Admin (one-to-many).
- An Exam includes many Questions through the Exam Question join table; a Question may appear in more than one Exam (many-to-many).
- A User with role Student has many Exam Attempts; an Exam Attempt belongs to exactly one Student and exactly one Exam (one-to-many).
- An Exam Attempt has many Answers, at most one per Question in that exam (one-to-many, unique per question).
- An Exam Attempt has many Violation Log Entries, each timestamped independently (one-to-many).

In compact notation the relationships are:

- `users 1---* exams (as creator)` and `users 1---* questions (as creator)`
- `exams *---* questions (via exam_questions)`
- `users 1---* exam_attempts *---1 exams`
- `exam_attempts 1---* answers *---1 questions`
- `exam_attempts 1---* violations`

6.3 Complete Data Dictionary

The data dictionary lists the fields of each table with their data type and key or constraint. Types follow

the entity-relationship diagram of the SRS companion document.

Table: users
User accounts for both roles. One account per person; the role flag distinguishes students from the admin.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the account.
full_name	string		Full name of the user.
email	string	UK (unique)	Email address or username used to log in; unique across all accounts.
password_hash	string		Salted password hash; the plain password is never stored or logged.
role	string		Role flag, student or admin, used for dashboard routing and authorization.
created_at	datetime		Time at which the account was created.

Table: questions
The question bank. Questions persist independently of any single exam and may be reused across exams.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the question.
text	string		Question text.
option_a	string		First answer option.
option_b	string		Second answer option.
option_c	string		Third answer option.
option_d	string		Fourth answer option.
correct_option	int		Index of the single correct option; never sent to the client while an attempt is in progress.
topic	string		Optional topic or category tag.
created_by	int	FK > users.id	The Admin who created the question.

Table: exams
Exam definitions created by the admin.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the exam.
title	string		Exam title.
duration_minutes	int	CHECK 5 to 180	Allowed duration of one attempt, in minutes.
available_from	datetime		Start of the scheduled availability window.

Field Name	Data Type	Key / Constraint	Description
available_to	datetime		End of the scheduled availability window.
created_by	int	FK > users.id	The Admin who configured the exam.

Table: exam_questions

Join table resolving the many-to-many relationship between exams and questions.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the link record.
exam_id	int	FK > exams.id	The exam that includes the question.
question_id	int	FK > questions.id	The question included in the exam.

Table: exam_attempts

One record per student per exam; the central record of the system.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the attempt.
exam_id	int	FK > exams.id	The exam being attempted.
student_id	int	FK > users.id	The student making the attempt.
started_at	datetime		Server-recorded start time; the deadline is derived from it.
submitted_at	datetime		Server-recorded submission time.
submission_reason	string		How the attempt was submitted: manual or auto.
score	int		Score computed by the server after submission.

Table: answers

The student's selected option per question within an attempt.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the answer record.
attempt_id	int	FK > exam_attempts.id	The attempt to which the answer belongs.
question_id	int	FK > questions.id	The question answered.
selected_option	int		The option selected by the student; a changed selection overwrites the previous one.
answered_at	datetime		Time at which the answer was recorded.

Table: violations

The violation log: one entry per reported tab-switch event.

Field Name	Data Type	Key / Constraint	Description
id	int	PK	Unique identifier of the violation entry.
attempt_id	int	FK > exam_attempts.id	The attempt during which the event occurred.
event_type	string		Client-observed event type (hidden or visible).
occurred_at	datetime	server-assigned	Timestamp assigned by the server on receipt, never taken from the client.

6.4 Data Retention

- Exam attempts, answers and violation logs are retained for the academic term or project evaluation period and are not deleted automatically.
- Question bank entries persist independently of any single exam and may be reused across exams.
- On account removal, a student's past attempts and violation logs are retained in an anonymised form for reporting continuity, rather than being deleted outright.

6.5 Data Integrity

- Unique constraint on `users.email`.
- Unique compound constraint on `answers (attempt_id, question_id)`, preventing more than one stored answer per question within an attempt.
- Unique compound constraint on `exam_attempts (exam_id, student_id)` for attempts that are active or submitted, enforcing the single-attempt rule (FR-EXAM-06).
- Check constraint ensuring `exams.duration_minutes` falls within the permitted 5 to 180 minute range.
- Foreign key constraints with restricted deletion on `exam_questions`, `answers` and `violations`, so that a referenced exam, question or attempt cannot be removed while dependent records exist.
- `violations.occurred_at` is always set from the server clock at the time the event is received, never from a client-supplied value.

7. SECURITY & ACCESS CONTROL MATRIX

7.1 Role-Based Access Control (RBAC)

Authentication is shared by both roles, but authorization is evaluated on every protected request. A valid login is not sufficient when the requested resource belongs to another student or requires the admin role. The matrix below summarises who may perform which action.

System Action	Unauthenticated Guest	Authenticated Student	Admin
Log in	Yes	n/a	n/a
Register account	Self-registration, if enabled	No	Provision accounts
Log out	n/a	Yes	Yes
List exams	No	Exams within availability window	All exams
Start and take an exam	No	Yes (one attempt per exam)	No
Submit an attempt (manual or auto)	No	Own attempt only	No
View own score	No	Yes, after scoring	n/a
View another student's result	No	No	Yes
View answer key (correct option)	No	No	Yes, in the question bank
Create, edit or remove questions	No	No	Yes (locked while attempts in progress)
Configure exams	No	No	Yes
View violation count and log	No	No (hidden during and after the attempt in the student interface)	Yes, in the admin report
Export report as CSV	No	No	Yes

7.2 Server-Side Security Controls

Security is implemented as a set of small controls placed at the request boundary, the service boundary and the persistence boundary. The table lists each control and the threat or failure it reduces.

Security Control	Threat or Failure Reduced
HTTPS outside local loopback	Reduces interception of credentials, session tokens and exam requests in non-local deployment.
Salted password hashing	Prevents plaintext password persistence and reduces damage from a database disclosure.

Security Control	Threat or Failure Reduced
Token, role and ownership checks	Prevents unauthenticated access, privilege escalation and cross-student data access.
Server-side validation and ORM binding	Reduces malformed input, SQL injection and invalid state transitions.
Answer-key protection	The correct option is never sent to the browser while an attempt is in progress.
Server-side time and timestamps	Prevents client clock drift from extending exams or falsifying violation history.
Violation count concealment	Prevents a student from calibrating how much tab-switching is tolerated.
Scope limitation	No webcam, audio or screen recording, and no claim of complete malpractice prevention.

Request-processing order

Every protected request follows the same order, so that no business operation runs before the caller has been authenticated and authorized:

1. Validate the session token (HTTP 401 if missing or invalid).
2. Check that the account's role is permitted for the endpoint (HTTP 403 otherwise).
3. Check resource ownership, for example that an attempt belongs to the requesting student.
4. Validate the payload server-side and bind it through the ORM.
5. Execute the business operation (exam, proctoring, scoring or reporting service).
6. Persist through the data-access layer and return a small JSON response.

7.3 Security Scope and Residual Risks

The academic build is **not presented as a public-internet hardened system**. Tab-switch detection is a deterrent and evidentiary signal. It can be bypassed with a second physical device, a secondary monitor outside the browser's visibility model, a virtual machine, or by disabling JavaScript. Any deployment outside the classroom or lab context would require an additional security review, in particular against the OWASP Top 10 web application risks listed in the references.

8. COMPREHENSIVE TEST SUITE

8.1 Verification Strategy

Verification is planned at four levels. The requirements in Chapters 3 and 4 are written with explicit inputs, processes, outputs and error codes, so that test cases follow from them directly. The SRS states that the core logic (scoring, timer-expiry enforcement and violation logging) is to be covered by automated tests, and the architecture document names five areas on which testing should concentrate: timer-expiry enforcement, single-attempt enforcement, score calculation, authorization ownership and server-side violation logging.

- **Unit Testing:** validation rules (password policy, exactly one correct option, duration between 5 and 180 minutes), score calculation, and deadline computation from the server start time.
- **Integration Testing:** interaction between the Flask endpoints, the service layer and the database, for example attempt start with the single-attempt rule, answer upsert, rejection of late answers, and violation logging with a server timestamp.
- **End-to-End System Testing:** complete user journeys in a browser, from login through exam, submission and result, and from the admin's question creation to the consolidated report and CSV export.
- **Security and Non-Functional Testing:** authorization checks (HTTP 401 and 403), absence of the answer key in client payloads, timer accuracy, response-time targets and lab-scale concurrency.

The test cases below are a **test plan**. The Status column is deliberately left as "Not yet executed": actual results must be recorded by the student after running each case, and must not be filled in beforehand.

8.2 Functional Test Cases

Test ID	Requirement	Scenario Description	Expected Result	Status
TC-AUTH-001	FR-AUTH-01	Register a student with a valid name, unique email and a password meeting the policy.	Account created in active state; password stored only as a salted hash; acknowledgement returned.	Not yet executed
TC-AUTH-002	FR-AUTH-01	Register with an email that already exists.	Request rejected with HTTP 409; no second account created.	Not yet executed
TC-AUTH-003	FR-AUTH-01	Register with a password below the policy (fewer than 8 characters or missing mixed case or digit).	Request rejected with HTTP 422.	Not yet executed
TC-AUTH-004	FR-AUTH-02	Log in as a student with correct credentials.	Session token and role (student) returned; user routed to the Student Dashboard.	Not yet executed
TC-AUTH-005	FR-AUTH-02	Log in as the admin with correct credentials.	Session token and role (admin) returned; user routed to the Admin Dashboard.	Not yet executed
TC-AUTH-006	FR-AUTH-02	Log in with a wrong password.	HTTP 401; no session token issued.	Not yet executed
TC-AUTH-007	FR-AUTH-03	Log out, then reuse the old token on a protected endpoint.	Token invalidated; the request is rejected with HTTP 401.	Not yet executed

Test ID	Requirement	Scenario Description	Expected Result	Status
TC-AUTH-008	FR-AUTH-04	A student calls an admin-only endpoint with a valid token.	HTTP 403; handler does not execute.	Not yet executed
TC-QB-001	FR-QB-01	Admin creates a question with four options and exactly one correct option.	Question stored; identifier returned.	Not yet executed
TC-QB-002	FR-QB-01	Admin submits a question with no correct option, or with more than one.	HTTP 422; nothing stored.	Not yet executed
TC-QB-003	FR-QB-02	Admin edits a question that belongs to an exam with attempts in progress.	HTTP 409; question unchanged.	Not yet executed
TC-QB-004	FR-QB-03	Admin configures an exam with a valid question set, a 30-minute duration and an availability window.	Exam stored; identifier and summary returned.	Not yet executed
TC-QB-005	FR-QB-03	Admin configures an exam with an empty question set, or with a duration outside 5 to 180 minutes.	HTTP 422; exam not stored.	Not yet executed
TC-EXAM-001	FR-EXAM-01	Student starts an exam inside its availability window.	Attempt created with a server start time; question set returned without the correct-answer indicator; remaining time returned.	Not yet executed
TC-EXAM-002	FR-EXAM-01	Student starts an exam outside its availability window.	HTTP 403; no attempt created.	Not yet executed
TC-EXAM-003	FR-EXAM-01 / 06	Student starts the same exam a second time, including from a second tab or device.	HTTP 409; only one attempt record exists.	Not yet executed
TC-EXAM-004	FR-EXAM-02	Student selects an option, then changes it.	Only one answer record exists for the question and it holds the latest selection.	Not yet executed
TC-EXAM-005	FR-EXAM-02 / 04	An answer is sent after the server-computed deadline.	Request rejected (HTTP 409); the attempt is treated as due for submission.	Not yet executed
TC-EXAM-006	FR-EXAM-03	Refresh the exam page mid-attempt.	Remaining time is recomputed from the server start time; it is neither reset nor extended.	Not yet executed
TC-EXAM-007	FR-EXAM-04	Let the countdown reach zero with only some questions answered.	Attempt submitted automatically with the recorded answers; submission reason recorded as auto.	Not yet executed
TC-EXAM-008	FR-EXAM-04	Close the tab before the deadline and never send the client auto-submit request.	After the deadline the server still treats the attempt as expired and submitted; it is not left unresolved.	Not yet executed

Test ID	Requirement	Scenario Description	Expected Result	Status
TC-EXAM-009	FR-EXAM-05	Student submits manually, then tries to submit or answer again.	First submission accepted with reason manual; later requests rejected with HTTP 409.	Not yet executed
TC-PROC-001	FR-PROC-01 / 03	During an active attempt, switch to another tab or window and return.	A violation entry is stored for the attempt with a server-assigned timestamp.	Not yet executed
TC-PROC-002	FR-PROC-03	Send a violation report with a client-supplied time far from the server time.	Stored `occurred_at` equals the server's receipt time; the client time is not used.	Not yet executed
TC-PROC-003	FR-PROC-03	Send a violation report for an attempt that is already submitted.	Report not appended to the log for an inactive attempt.	Not yet executed
TC-PROC-004	FR-PROC-02	Close the exam tab during an active attempt.	A final event is sent with the Beacon API on a best-effort basis; delivery is not guaranteed in every browser.	Not yet executed
TC-PROC-005	FR-PROC-04	Inspect all responses and screens shown to a student during the attempt.	The running violation count appears nowhere in the student interface or responses.	Not yet executed
TC-RES-001	FR-RES-01	Submit an attempt with a known set of correct and incorrect answers.	The stored score equals the number of answers matching the answer key.	Not yet executed
TC-RES-002	FR-RES-02	A student opens their result after submission.	Score, total questions and submission time are shown; no violation log and no other students' results.	Not yet executed
TC-RES-003	FR-RES-03	Admin opens the consolidated report for an exam.	One row per attempting student with score, submission reason, violation count and the full timestamped violation log.	Not yet executed
TC-RES-004	FR-RES-04	Admin exports the report.	A CSV file downloads with one row per attempt and its score and violation count.	Not yet executed
TC-ADM-001	FR-ADM-01	Admin opens the dashboard.	Counts of configured exams, attempts in progress and submitted attempts awaiting review are shown.	Not yet executed
TC-SEC-001	NFR Security	Inspect the question payload sent to the browser during an attempt.	The correct-answer indicator is absent from the payload.	Not yet executed
TC-SEC-002	NFR Security	A student requests another student's attempt or result.	Request rejected by the ownership check.	Not yet executed

8.3 Non-Functional Verification Plan

Quality Attribute	Target (from Chapter 4)	Suggested Verification Method	Status
API response time	Reads under 500 ms; answer capture and submission under 1 s	Time repeated calls under normal lab-scale load	Not yet executed
Exam interface load	Interactive within 2 seconds on standard broadband	Measure page load in the browser developer tools	Not yet executed
Timer accuracy	Within 2 seconds of the server-computed deadline	Compare the displayed countdown with the server value over a full exam duration	Not yet executed
Violation delivery	Within 1 second of detection, network permitting	Compare client detection time with server receipt time	Not yet executed
Concurrency	At least 50 concurrent attempts	Run a scripted load of 50 simultaneous attempts	Not yet executed
Reliability	Refresh or brief network drop does not change remaining time	Refresh and disconnect mid-attempt and compare remaining time	Not yet executed

8.4 Test Execution Record

After testing, record the measured outcome of each case in this report. Insert a screenshot of the automated test run and of the browser checks below.

[IMAGE PLACEHOLDER – REPLACE THIS BOX]

Insert: a screenshot of your automated test run (for example the terminal output of the test runner) and a completed results table

Do not fabricate results: insert only the output of tests that you have actually run.

Delete this box after inserting the picture (Insert > Pictures). Do not leave this placeholder in the final report.

Figure S.3: Automated test run output and recorded test results

9. PROJECT TIMELINE & SDLC

9.1 SDLC Methodology

The project follows the standard software development life cycle. The SRS bounds the build to a **six-week academic development timeline**, which in turn bounds the feature set to the requirements listed in Chapter 3. Each phase of the life cycle produced, or is expected to produce, a defined artifact, as shown below.

Phase	Activity	Artifact in this Report
1. Planning	Topic approval, feasibility and objectives, recorded in the college minor project proposal.	Project proposal (referenced in Section 1.5)
2. Requirements	Analysis of the gap in unmonitored online exams; specification of functional and non-functional requirements.	Part I, Chapters 1 to 7
3. Design	Translation of the requirements into UML diagrams and a three-layer architecture.	Part II (UML) and Part III (Architecture)
4. Implementation	Development of the HTML/CSS/JavaScript client, the Flask REST service and the SQLAlchemy data layer.	Source code on GitHub; Part IV screenshots
5. Testing	Execution of the test plan and recording of results.	Chapter 8
6. Documentation & Release	Final report, setup documentation and public repository for reproducibility.	This report and the GitHub repository

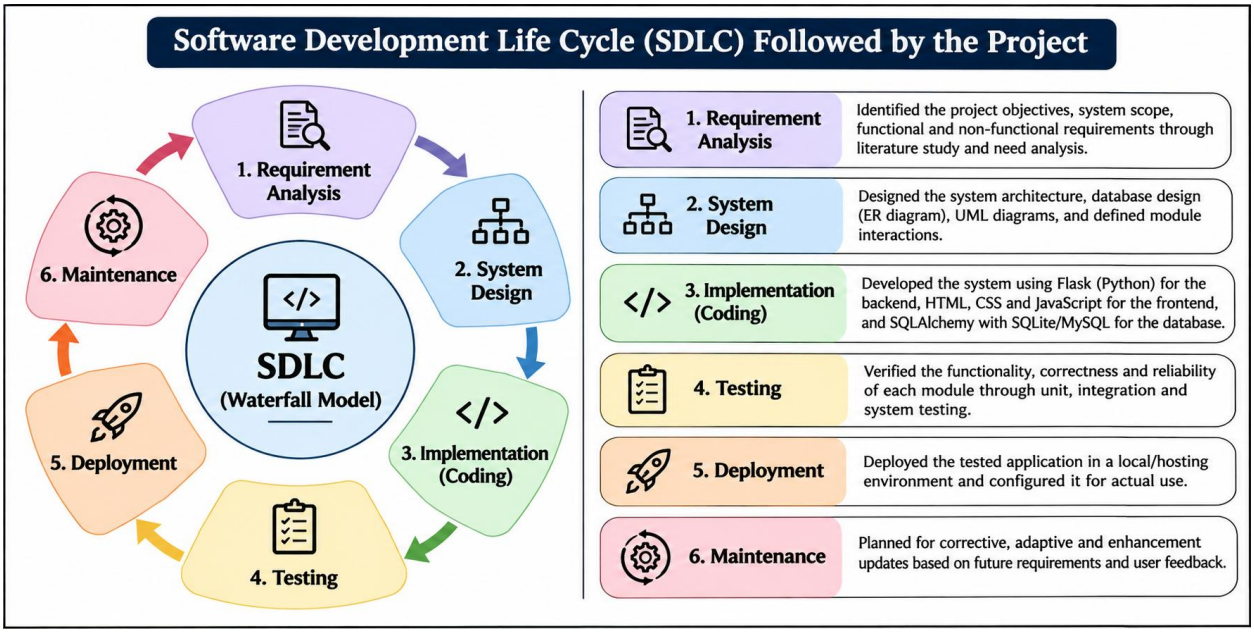


Figure S.4: Software development life cycle followed by the project

9.2 Milestone Schedule

The SRS fixes only the overall six-week scope; it does not give week-by-week dates. The duration and timeline of each phase must therefore be entered by the student from the actual project schedule.

Phase	Development Task	Duration	Timeline
1	Topic approval and feasibility analysis	[to be filled]	[to be filled]
2	Requirements gathering and SRS	[to be filled]	[to be filled]
3	UML and architecture design	[to be filled]	[to be filled]
4	Implementation (client, Flask service, database)	[to be filled]	[to be filled]
5	Testing and bug fixing	[to be filled]	[to be filled]
6	Final documentation and GitHub release	[to be filled]	[to be filled]
	Total (as scoped in the SRS)	**Six weeks**	

10. LIMITATIONS, FUTURE SCOPE & CONCLUSION

10.1 System Limitations

The following limitations are part of the specified design. They must be kept in mind when interpreting the output of the system.

- **Detection scope:** the system detects only browser tab or window visibility changes. It does not perform webcam-based face detection, audio monitoring or screen recording.
- **Circumventability:** tab-switch detection can be defeated by a student using a second physical device, a secondary monitor outside the browser's visibility model, a virtual machine, or by disabling JavaScript. The system is a deterrent and evidentiary signal, not a guarantee of exam integrity.
- **Single-institution scope:** the system is built for one institution or class context at a time; it is not a multi-tenant SaaS product.
- **Mobile browsers out of scope:** visibility-change semantics differ across mobile browsers and operating-system app switching, so the exam interface targets desktop browsers only.
- **Academic project timeline:** the system is scoped to a six-week development timeline (Section 6 of the project proposal), which bounds the feature set to what is listed in Chapter 3.
- **No answer-similarity or plagiarism detection:** only tab-switch behaviour is monitored; content-based cheating detection is out of scope.

10.2 Future Scope

The items below are **explicitly out of scope for this build** and are recorded only as a stated direction for extension. None of them is part of the specified system.

Future Feature	Possible Technology	Purpose	Status
Webcam-based face detection	OpenCV	Flag when the student's face is not visible during the attempt.	Proposed, not in this build
Randomised question order and difficulty-tiered question banks	Application logic over the existing question bank	Reduce answer sharing and allow tiered assessments.	Proposed, not in this build
PDF export of the proctoring report	To be selected	Provide a PDF alongside the CSV export already in scope.	Proposed, not in this build
Multiple-camera or audio-based anomaly detection	To be selected	Stronger AI-assisted proctoring.	Proposed, not in this build

10.3 Conclusion

The AI-Based Online Exam Proctoring System specifies a small, well-bounded platform for timed MCQ assessment in which the integrity-sensitive rules live on the server. Time is derived from the server-recorded start time, scoring is performed on the server, and every violation is timestamped by the server, so that neither a client clock nor a manipulated client can distort the record. The student does not see their own violation count during the attempt, and the admin sees the count and the full

timestamped log next to the score.

The system fills a real gap: unmonitored online quizzes record answers but give the evaluator no signal about whether the student stayed on the exam page. By capturing the one behaviour that is detectable without special hardware, the system gives a human reviewer evidence that can be weighed rather than left unrecorded. At the same time, this report is explicit that the signal is narrow. It is a lightweight deterrent and reporting signal, not complete AI-based cheating detection, and it is not a guarantee of exam integrity.

Technical Architecture & Core Capabilities

- **Client:** HTML5, CSS3 and vanilla JavaScript (ES6+) running in a desktop browser; login, dashboards, exam interface, countdown timer, result view, and the visibility listener.
- **Application backend:** Python 3.10+ Flask REST service with route handlers, middleware for token, role and ownership checks, and separate services for exams, proctoring, scoring and reporting.
- **Data layer:** SQLAlchemy ORM over SQLite (development) or MySQL (shared deployment), with unique, check and foreign-key constraints protecting integrity.
- **Proctoring signal:** Page Visibility API for tab-switch detection and the Beacon API for best-effort delivery of a final event on tab close, with server-assigned timestamps.
- **Security:** salted password hashing, session tokens with idle expiry, role and ownership checks, answer-key protection and HTTPS outside local loopback.
- **Extensibility:** the modular service structure leaves room for the proposed future features without redesigning the core.

UML DIAGRAM DOCUMENT

This part of the report presents the **UML and database diagrams** of the AI-Based Online Exam Proctoring System. It translates the Software Requirements Specification into simplified structural, behavioural, architectural and data diagrams. Each diagram is described on its own page so that the relationships remain easy to follow. The diagrams are shown as placeholders in this document and must be replaced with the final diagrams.

TABLE OF CONTENTS

Sr. No.	Section / Diagram
1	USE CASE DIAGRAM
2	CLASS DIAGRAM
3	SEQUENCE DIAGRAM – AUTHENTICATION
4	SEQUENCE DIAGRAM – EXAM ATTEMPT
5	SEQUENCE DIAGRAM – PROCTORING
6	COMPONENT DIAGRAM
7	ER DIAGRAM – DATABASE STRUCTURE
8	ACTIVITY DIAGRAM – EXAM LIFECYCLE
	CONCLUSION

1. USE CASE DIAGRAM

1.1 Introduction

The Use Case Diagram shows the actors of the system and the main capabilities available to each of them. The system has **two actors**, the Student and the Admin. Both must pass through the common role-based login and session use case, after which each actor reaches a different set of functions. This mirrors the role-based authorization required by FR-AUTH-04: the same login mechanism serves both roles and the role flag on the account decides which dashboard and which functions are available.

Actors

- **Student:** takes a timed exam, answers questions, submits the attempt and views only their own result.
- **Admin:** manages the question bank and exams and reviews the consolidated score and violation report.

Main Student Use Cases

- **Role-based Login / Session:** authenticate and receive a session token (shared with the Admin).
- **View available exams:** list the exams that are inside their availability window.
- **Start timed exam:** begin the single permitted attempt on an exam.
- **Answer MCQs:** select an option for each question; a changed selection overwrites the earlier one.
- **Tab-switch events logged:** during an active attempt, every departure from the exam tab is detected and logged. This is shown in the diagram as a dotted relationship labelled "active attempt", because it happens automatically as part of the attempt and is not an action chosen by the student.
- **Manual or auto-submit:** submit before time expires, or be submitted automatically when the timer reaches zero.
- **View own result:** see the score, total questions and submission time, and nothing else.

Main Administrator Use Cases

- **Manage question bank:** create, edit and remove MCQ items.
- **Configure exam:** choose questions, set the duration and set the availability window.
- **Review score + violations:** open the consolidated per-student report.

Export CSV: download the report for record-keeping.

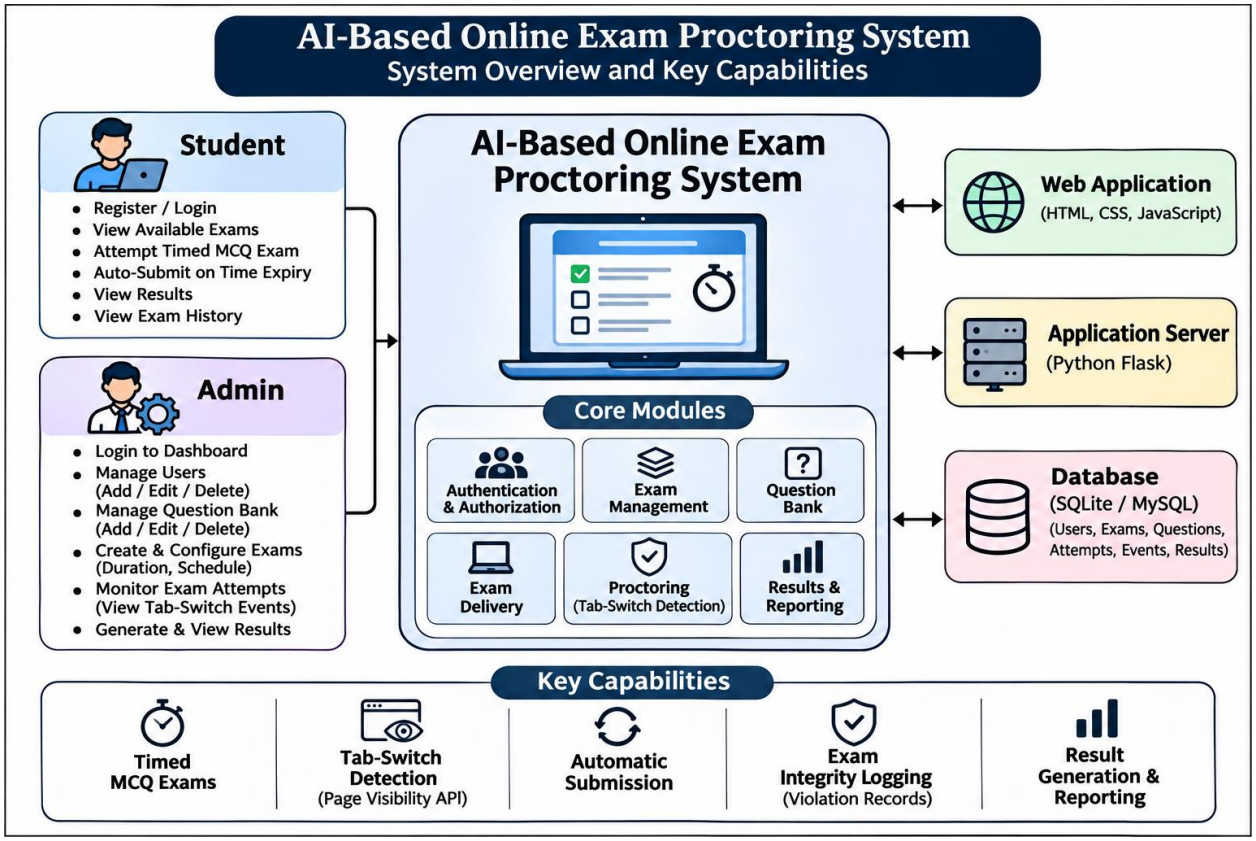


Figure 1: Use Case Diagram of the AI-Based Online Exam Proctoring System

2. CLASS DIAGRAM

2.1 Introduction

The Class Diagram shows the core domain classes and the service classes that coordinate authentication, exams, proctoring and reporting. The central domain class is the **ExamAttempt**. It belongs to one student and one exam, records answers, receives violation entries, and is scored after manual or automatic submission. The domain classes correspond one-to-one with the database tables described in Chapter 6, while the service classes correspond to the application services of the architecture in Part III.

Main Domain Classes

Class	Attributes	Operations
User	id, full_name, email, password_hash, role, created_at	login(), logout()
Question	id, text, option_a, option_b, option_c, option_d, correct_option, topic	create(), edit()
Exam	id, title, duration_minutes, available_from, available_to	publish()
ExamQuestion	id, exam_id, question_id	(link class)
ExamAttempt	id, started_at, submitted_at, submission_reason (SubmissionReason), score	start(), submit(), calculateScore()
Answer	id, selected_option, answered_at	saveOrUpdate()
Violation	id, event_type, occurred_at	recordServerTimestamp()

Service Classes

Service	Operations	Responsibility
AuthService	authenticate(), authorizeRole()	Verify credentials and enforce role checks.
ExamService	startAttempt(), captureAnswer(), enforceDeadline(), autoSubmit()	Manage the attempt lifecycle and the server-side deadline.
ProctoringService	receiveVisibilityEvent(), logViolation()	Receive visibility events and append server-timestamped violations.
ReportingService	scoreAttempt(), buildAdminReport(), exportCSV()	Score attempts and build the admin report and CSV export.

Main Relationships

- An Admin (User) creates Questions and configures Exams.
- An Exam contains Questions through ExamQuestion; a Question can appear in several Exams.
- A Student (User) owns ExamAttempts; an Exam receives ExamAttempts.

- An ExamAttempt records Answers (each answer is for one Question) and logs Violations.

ExamService manages attempts and captures answers; ProctoringService records violations; ReportingService reads attempts and summarises violations.

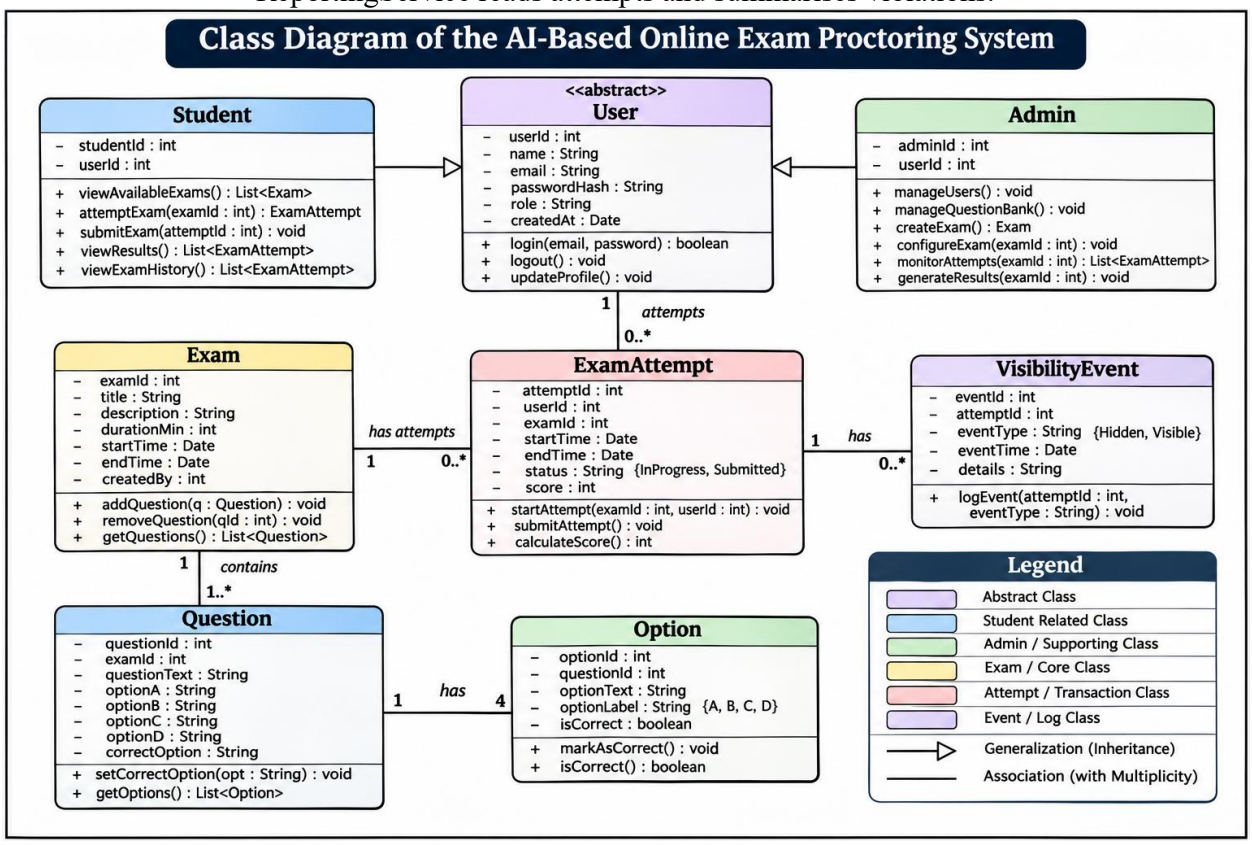


Figure 2: Class Diagram of the AI-Based Online Exam Proctoring System

3. SEQUENCE DIAGRAM – AUTHENTICATION

3.1 Introduction

This sequence diagram shows login, session-token creation, role routing, protected-request checks and logout. The backend validates credentials and role permissions on every protected request. Passwords are verified against a stored hash; the plain password is never persisted.

Participants

- User
- Web Browser
- Flask Auth API
- SQL Database

Authentication Flow

- The user enters an email and password in the browser.
- The browser sends `POST /login` to the Flask Auth API.
- The API looks up the user by email in the SQL database.
- The database returns the user record with the password hash and role.
- The API verifies the password and creates a session token.
- The API replies `200 OK` with the token and role.
- The browser routes the user to the Student or Admin dashboard.
- **Loop, for every protected request:** the browser sends the request with the session token; the API validates the token and role.
- **If authorized:** the requested operation is executed and the result returned. **If the token is invalid or the role is wrong:** a 401 or 403 error is returned.

On logout the browser sends `POST /logout`; the API invalidates the session in the database and confirms that the session has ended.

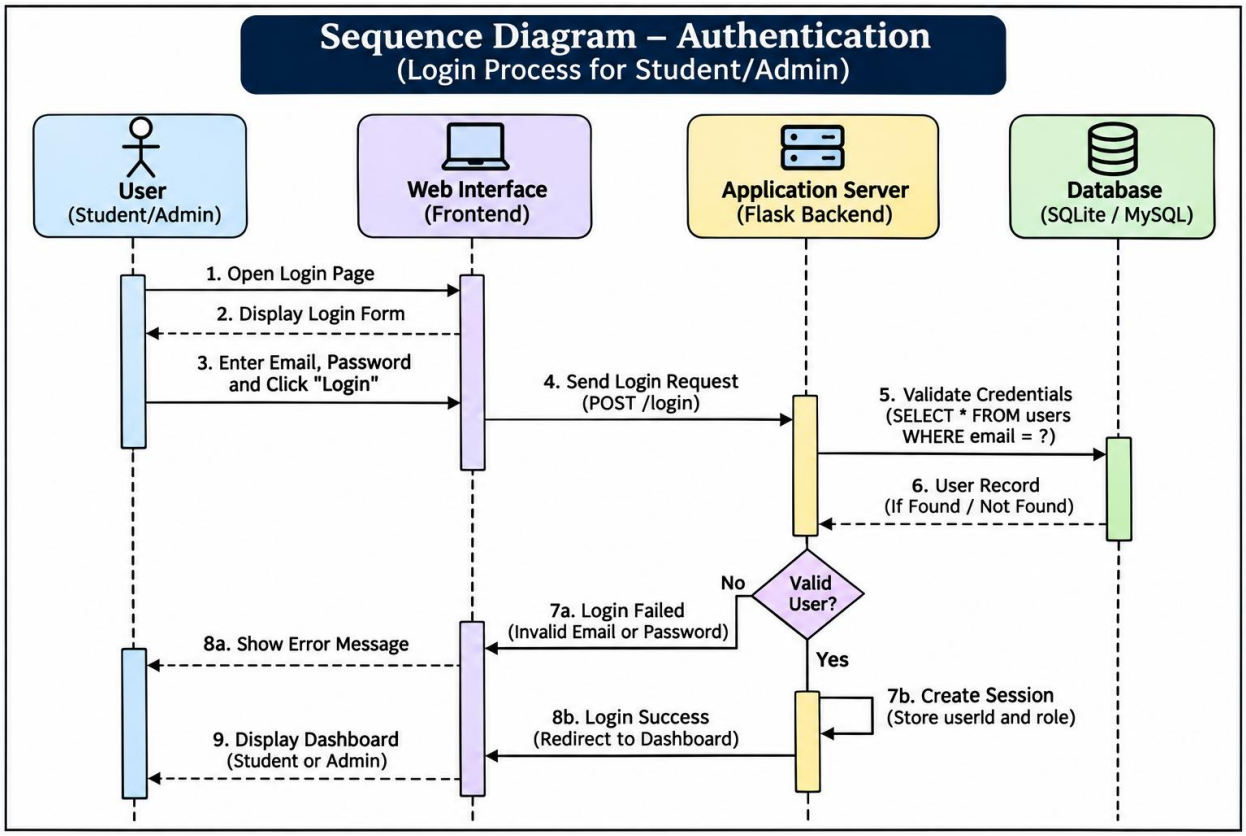


Figure 3: Sequence Diagram – Authentication

4. SEQUENCE DIAGRAM – EXAM ATTEMPT

4.1 Introduction

This sequence diagram shows how an exam is started, how answers are captured, how the server deadline is enforced and how the submission is scored. The client displays the countdown, but the Flask backend remains the authority for the deadline. The server rejects late answer requests and records whether the submission was manual or automatic.

Participants

- Student
- Exam Browser UI
- Flask Exam API
- SQL Database

Exam-Attempt Flow

- The student selects an available exam.
- The browser sends `POST /attempts/start`; the API checks availability and the active-attempt rule against the database.
- **If valid to start:** the API creates the attempt with a server start time and returns the questions, the server deadline and the attempt ID; the browser shows the exam and countdown. **If the exam is not available or an attempt already exists:** a 403 or 409 error is returned.
- **Loop, while the attempt is active:** the student selects an option; the browser sends `PUT /answers`; the API checks ownership and the server deadline, upserts the answer, confirms that the answer is saved and returns the reconciled remaining time.
- **If the student submits before the deadline:** the student confirms the manual submit and the browser sends `POST /attempts/submit`.
- **If the deadline is reached:** the browser sends an auto-submit request and the API enforces the deadline independently of the client.
- The API marks the attempt as submitted and stores the submission reason in the database.

The API returns the score and submission summary, and the browser shows the result.

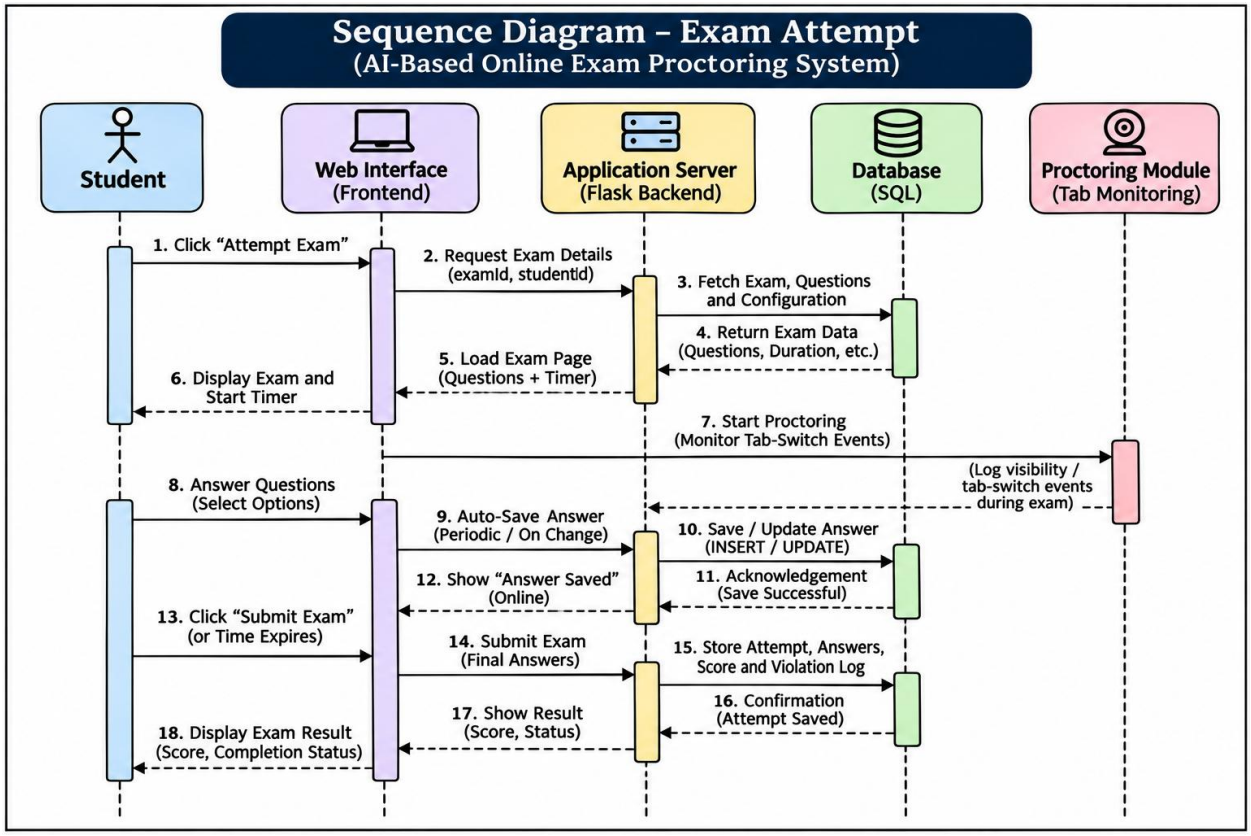


Figure 4: Sequence Diagram – Exam Attempt

5. SEQUENCE DIAGRAM – PROCTORING

5.1 Introduction

This sequence diagram shows browser visibility events, Beacon delivery on tab close and server-side violation logging. Only the tab or window visibility signal is in scope. The server assigns the event timestamp and keeps the running violation count hidden from the student during the attempt.

Participants

- Student
- Exam Page
- Flask Proctoring API
- SQL Database

Proctoring Flow

1. The `visibilitychange` listener is active only during an attempt.
2. The student switches away from the exam tab or window; `document.hidden` becomes true.
3. The exam page sends `POST /violations` with the attempt ID and the event type.
4. The API verifies that the attempt is active, appends a violation with the server timestamp, and the database confirms that the violation is saved.
5. The API acknowledges the report to the exam page.
6. If the student closes the tab or window, the page calls `navigator.sendBeacon` with a final event.
7. The API timestamps the event on receipt and appends the final violation if the attempt is still active.

Two notes in the diagram record the design intent: the running violation count is **not** sent to the student, and the admin report later reads the full timestamped log. Delivery of the final Beacon event is best-effort and is not guaranteed in every browser.

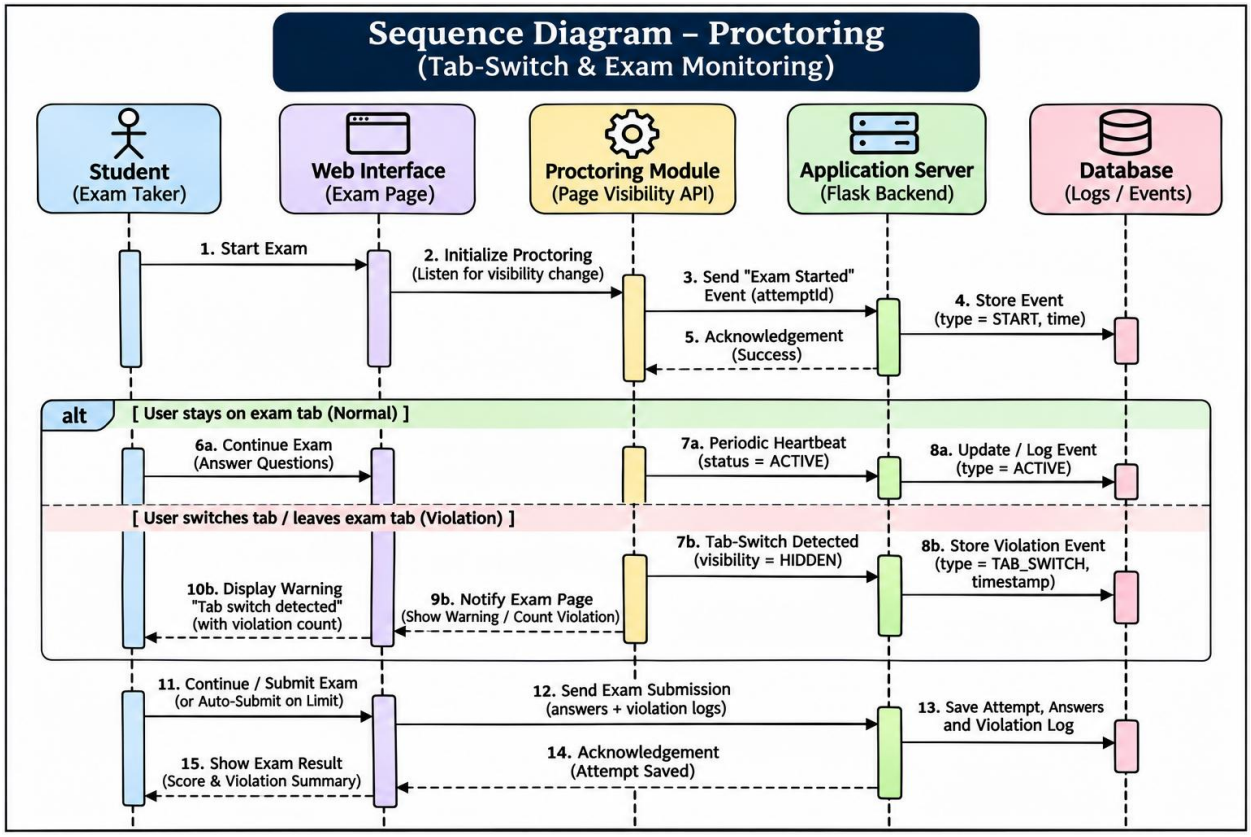


Figure 5: Sequence Diagram – Proctoring

6. COMPONENT DIAGRAM

6.1 Introduction

The Component Diagram shows the logical three-layer architecture: the browser client, the Flask application and the relational database. Student and admin interfaces call the REST API with JSON and a session token. SQLAlchemy centralises data access, while the Page Visibility API and the Beacon API provide browser-level signals.

Main Components

Client Layer – Desktop Browser

- Student UI: login, dashboard, exam, result
- Admin UI: dashboard, question bank, report
- Page Visibility API: `visibilitychange` / `document.hidden`
- Beacon API: final event on tab close (best effort)

Application Layer – Python Flask REST API

- Authentication & Authorization
- Exam Attempt & Answer Capture
- Proctoring & Violation Logging
- Scoring & Reporting
- Question Bank & Exam Configuration

Data Layer

- SQLAlchemy ORM / data access shared by all application components
- SQLite for development or MySQL for deployment

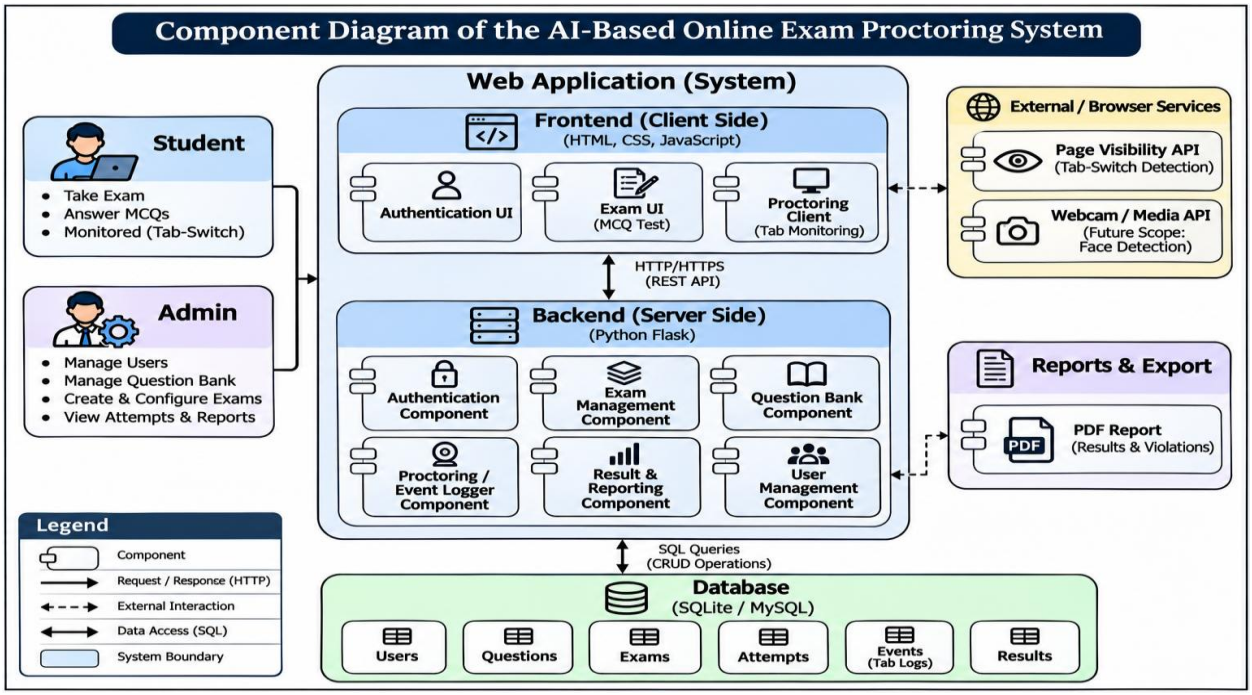


Figure 6: Component Diagram of the AI-Based Online Exam Proctoring System

7. ER DIAGRAM – DATABASE STRUCTURE

7.1 Introduction

7.2 The Entity-Relationship Diagram shows the database entities, their key fields, integrity markers and cardinalities taken from the SRS data model. The many-to-many relationship between Exam and Question is resolved through the ExamQuestions table. Unique constraints apply to the user email, to one answer per attempt and question, and to the single-attempt rule for a student and exam.

Entities and Important Fields

Entity	Important Fields and Markers
USERS	id (PK), full_name, email (UK), password_hash, role, created_at
QUESTIONS	id (PK), text, option_a to option_d, correct_option, topic, created_by (FK)
EXAMS	id (PK), title, duration_minutes (5-180), available_from, available_to, created_by (FK)
EXAM_QUESTIONS	id (PK), exam_id (FK), question_id (FK)
EXAM_ATTEMPTS	id (PK), exam_id (FK), student_id (FK), started_at, submitted_at, submission_reason, score
ANSWERS	id (PK), attempt_id (FK), question_id (FK), selected_option, answered_at
VIOLATIONS	id (PK), attempt_id (FK), event_type, occurred_at (server-assigned)

Main Relationships

- USERS creates QUESTIONS and configures EXAMS.
- USERS makes EXAM_ATTEMPTS; EXAMS receives EXAM_ATTEMPTS.
- EXAMS includes EXAM_QUESTIONS; QUESTIONS appears in EXAM_QUESTIONS.
- EXAM_ATTEMPTS records ANSWERS; QUESTIONS is answered for in ANSWERS.
  EXAM_ATTEMPTS logs VIOLATIONS.

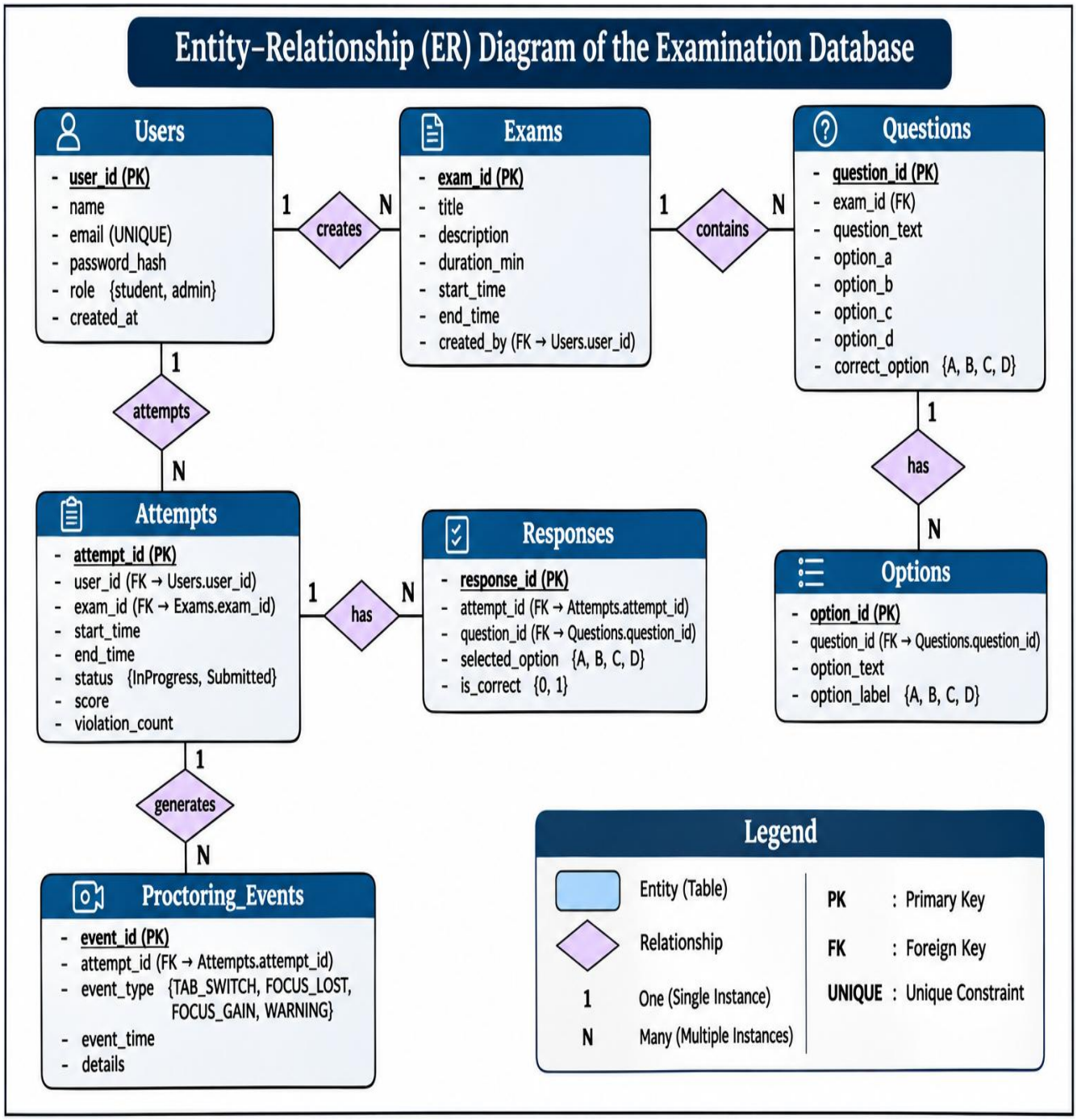


Figure 7: ER Diagram of the Examination Database

8. ACTIVITY DIAGRAM – EXAM LIFECYCLE

8.1 Introduction

The Activity Diagram shows the end-to-end operational flow for students and for the admin review process. During an active attempt, answer capture, timer checks and visibility-event logging occur together. Manual submission and deadline expiry converge at server-side scoring, after which the student sees the result and the admin can review the report.

Student Exam Lifecycle

1. Start, then Login, then the role decision (Student).
2. Student dashboard; decision: is an exam available? If not, return to the dashboard.
3. Start the timed attempt, answer MCQs and monitor the timer.
4. Decision: tab hidden? If yes, send a violation event; the attempt then continues.
5. Decision: continue? If the student chooses to stop, perform a manual submit.
6. Decision: deadline reached? If yes, the server-enforced auto-submit occurs; otherwise return to answering.
7. Server-side scoring, then the student views the result; End.

Admin Review Lifecycle

1. Start, then Login, then the role decision (Admin).
2. Admin dashboard, then manage questions and configure exams.
3. Review scores and violations, then an optional CSV export; End.

Decision Points

- What is the role of the logged-in user?
- Is an exam available?
- Was the tab hidden?
- Should the student continue or submit?

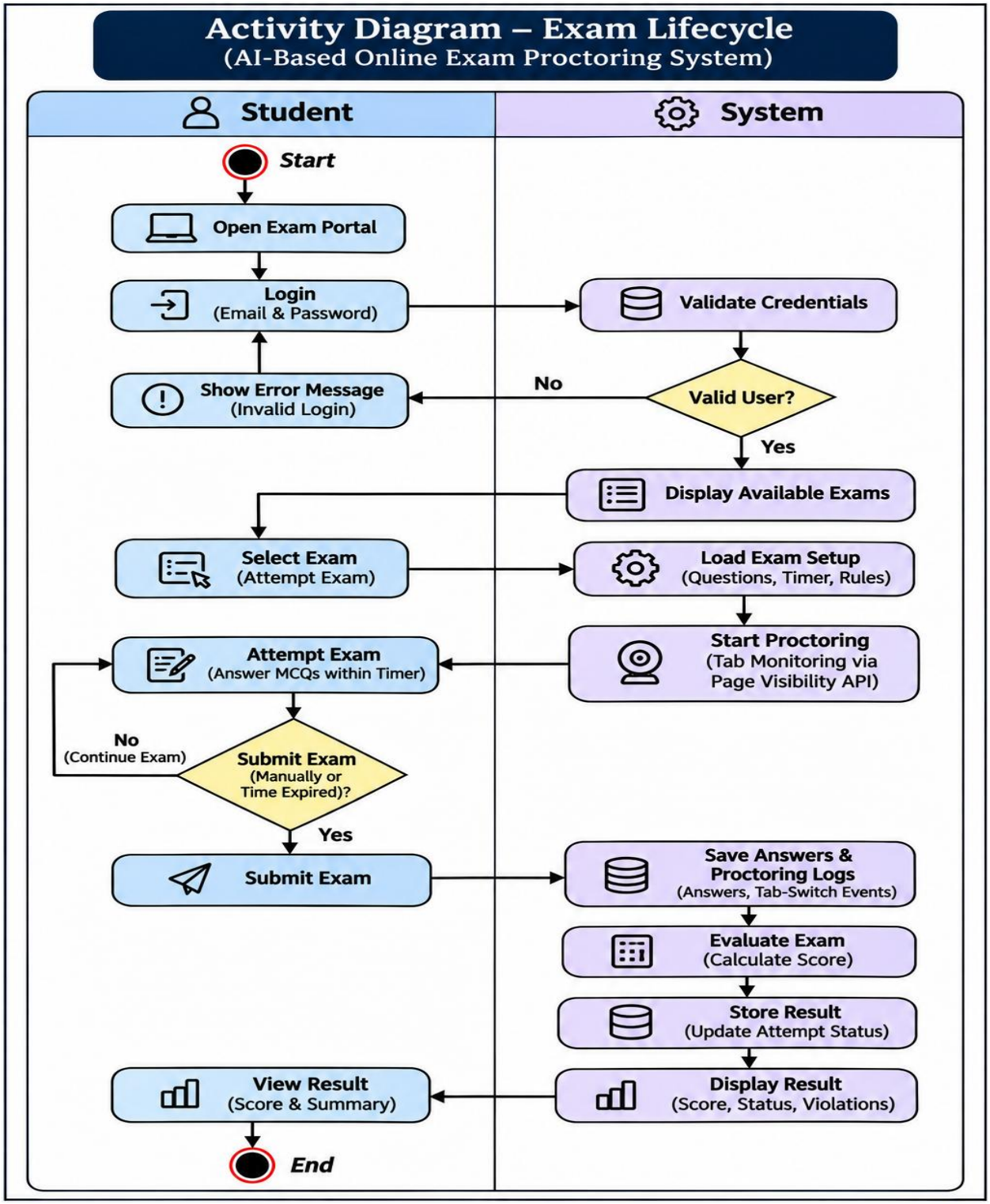


Figure 8: Activity Diagram – Exam Lifecycle

CONCLUSION

The eight diagrams describe the system from complementary viewpoints. The use-case diagram defines who does what; the class and ER diagrams define the domain and its storage; the three sequence diagrams define how authentication, the exam attempt and proctoring work step by step; the component diagram places everything in three layers; and the activity diagram connects the student and admin flows end to end.

All diagrams express the same architectural principle: the client observes and presents, the backend validates and decides, and the database preserves the record. The server owns the deadline, the score and the violation timestamps. The diagrams cover only the behaviour specified in the SRS, namely tab or window visibility proctoring, and do not depict features such as webcam monitoring that remain future scope.

ARCHITECTURE DESIGN DOCUMENT (ADD)

Document Version: 1.0

This Architecture Design Document converts the Software Requirements Specification of the AI-Based Online Exam Proctoring System into a simplified technical architecture. It focuses on responsibilities, interfaces, data ownership, security controls and deployment boundaries. It is intentionally limited to the academic build defined in the SRS.

TABLE OF CONTENTS

Sr. No.	Topic
1	INTRODUCTION AND GOALS
2	SYSTEM CONTEXT
3	OVERALL SYSTEM ARCHITECTURE
4	FRONTEND ARCHITECTURE
5	BACKEND ARCHITECTURE
6	DATABASE ARCHITECTURE
7	AUTHENTICATION AND AUTHORIZATION ARCHITECTURE
8	DATA FLOW PATTERNS
9	SECURITY ARCHITECTURE
10	DEPLOYMENT ARCHITECTURE
11	KEY DESIGN DECISIONS
12	FUNCTIONAL ARCHITECTURE
13	NON-FUNCTIONAL ARCHITECTURE
14	TECHNOLOGY STACK
15	REQUIREMENT-TO-ARCHITECTURE TRACEABILITY
16	FUTURE ARCHITECTURAL ENHANCEMENTS
17	ARCHITECTURAL SUMMARY
18	CONCLUSION

1. INTRODUCTION AND GOALS

1.1 Purpose

This architecture describes how the AI-Based Online Exam Proctoring System is divided into **browser, application and database responsibilities**. Its purpose is to show where each integrity-sensitive rule lives, how data moves between the layers, and which controls protect the exam record. Each architectural decision answers a risk or requirement stated in the SRS.

1.2 Project Overview

The system lets an institution run timed multiple-choice examinations online and records tab-switching behaviour during each attempt. A student logs in, attempts an exam, and is auto-submitted when the timer expires. The browser reports every departure from the exam tab to the server, which compiles a per-student violation log that the admin reviews together with the score.

1.3 Architectural Goals and Principle

Area	Architecture Statement
Primary goal	Deliver a timed MCQ examination with server-authoritative timing, answer capture, scoring and browser-level tab-switch logging.
Users	Student accesses assigned exams and own results. Admin manages questions, exams, accounts and consolidated reports.
Quality priorities	Integrity of the deadline, answer key, score and violation timestamps; clear separation of responsibilities; lab-scale reliability.
In scope	Role-based login, question bank, exam configuration, attempt lifecycle, auto-submit, violation log, scoring, admin report and CSV export.
Out of scope	Webcam or audio proctoring, face detection, browser lock-down, fullscreen enforcement, plagiarism detection and PDF export of reports.

Architecture principle: the client observes and presents; the backend validates and decides; the database preserves the record.

2. SYSTEM CONTEXT

2.1 System Context Description

The system is a self-contained web application used by two roles. Students and administrators interact through desktop browsers; the browser supplies visibility events, while the backend communicates with the relational database. The table lists every external party or interface and its responsibility.

External Party or Interface	Responsibility
Student	Logs in, starts one available exam, answers MCQs, submits, and views their own result.
Admin	Manages the question bank and exams, reviews all attempts and violation logs, and exports the consolidated report.
Desktop browser	Renders HTML/CSS/JavaScript screens, stores the session token, shows the timer, and sends JSON requests.
Page Visibility API	Provides the client-side signal when the exam tab becomes hidden or visible.
Beacon API	Attempts to deliver a final violation event when the page is closing.
SQLite / MySQL	Stores accounts, questions, exams, attempts, answers and violations.
Git / GitHub	Hosts versioned source and setup documentation for reproducibility.

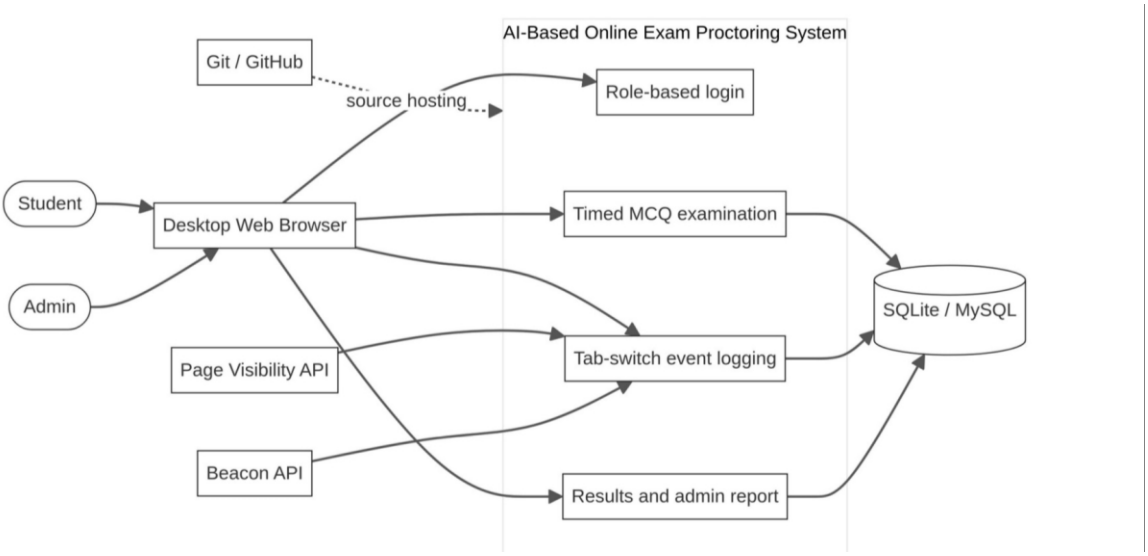


Figure 2.1: System context and external interfaces

3. OVERALL SYSTEM ARCHITECTURE

3.1 Architecture Style

The architecture follows a simple **layered model**. Requests pass from the browser to Flask services, then through SQLAlchemy to the database. No client-side score or deadline is trusted as authoritative.

3.2 Architecture Layers

Layer	Primary Responsibilities
Client layer	Login, dashboards, exam presentation, answer selection, countdown display, visibility listener and result screens.
Application layer	Authentication, role checks, exam delivery, deadline enforcement, answer capture, violation logging, scoring, reporting and configuration.
Data layer	Persistent relational records accessed through SQLAlchemy; foreign keys and unique constraints protect relationships.
Authority boundary	Server time controls expiry; server scoring controls results; server timestamps control violation history.
Integration style	REST-style JSON requests over HTTP locally and HTTPS for non-local deployment.

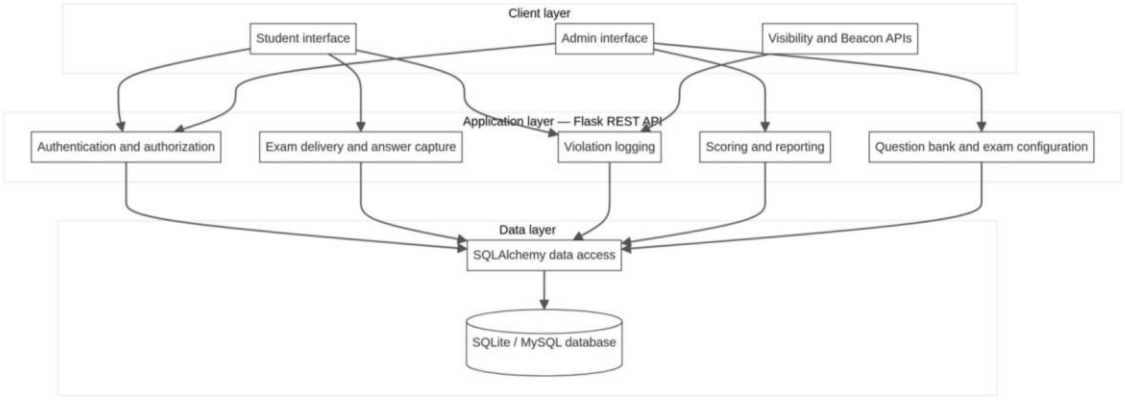


Figure 3.1: Overall three-layer architecture

4. FRONTEND ARCHITECTURE

4.1 Frontend Overview

The frontend is a vanilla **HTML5, CSS3 and JavaScript** client. It is responsible for interaction and display, while the server remains responsible for validation, timing, scoring and persistence.

4.2 Frontend Responsibilities

Frontend Area	Design Responsibility
Login screen	Collect username or email and password; store the returned session credential and route by role.
Student screens	List available exams, render MCQs, record selections locally until saved, show the countdown, and display the student's own result.
Admin screens	Show platform activity, question-bank forms, exam configuration, report viewer and CSV export action.
Timer service	Display remaining time from the server-computed deadline and request reconciliation periodically.
Proctoring listener	Attach `visibilitychange` only while an attempt is active; send a violation event without showing the running count to the student.

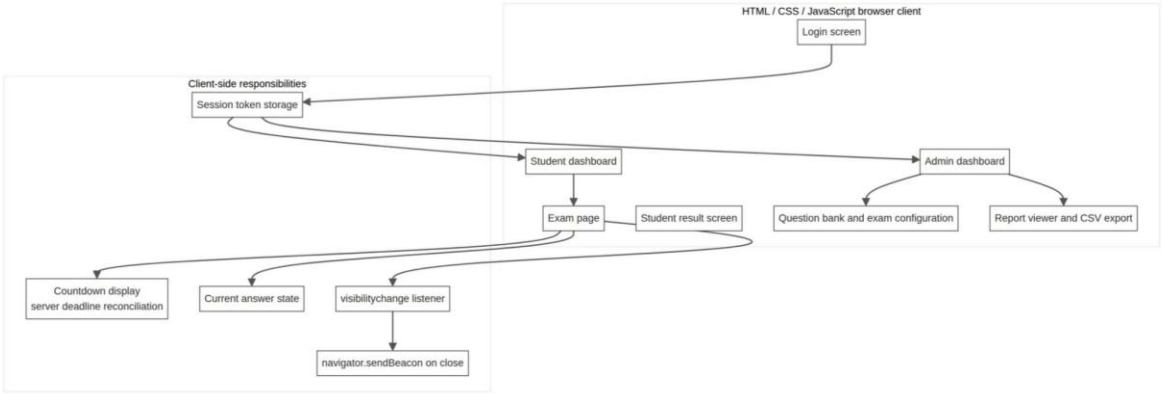


Figure 4.1: Frontend screens and client-side responsibilities

5. BACKEND ARCHITECTURE

5.1 Backend Overview

The backend separates request handling from business rules and data access. This prevents scoring, timer enforcement and violation logging from being duplicated across endpoints, and keeps each of them independently testable.

5.2 Backend Components

Backend Component	Responsibility
Route handlers	Parse JSON input, select the service operation and format HTTP responses.
Middleware	Validate session token, role, resource ownership and protected endpoint access before the handler executes.
Exam service	Start attempts, enforce the single-attempt rule, capture answers, reconcile deadlines and submit attempts.
Proctoring service	Validate active attempts and append server-timestamped visibility events.
Scoring / reporting	Compare submitted answers with the answer key, store scores, build admin reports and generate CSV output.
Repository layer	Centralise SQLAlchemy queries and transaction boundaries for consistent persistence.

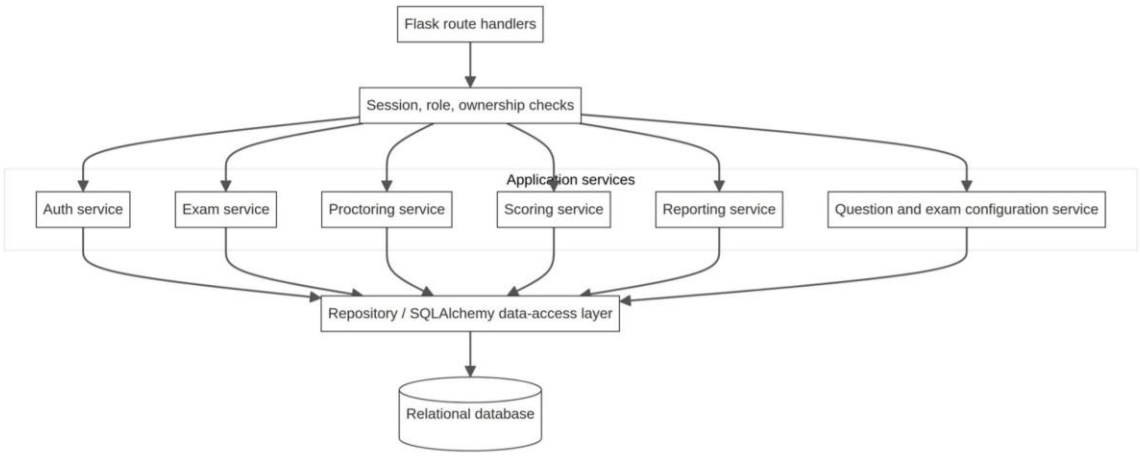


Figure 5.1: Backend service and data-access boundaries

6. DATABASE ARCHITECTURE

6.1 Database Overview

The database stores the durable academic record. Questions persist independently of one exam; attempts, answers and violations remain available for review and project evaluation. The table summarises the architectural rule for each data group.

6.2 Data Groups and Rules

Data Group	Architectural Rule
Identity	Users have unique email values, role information and salted password hashes; account removal preserves anonymised history.
Question and exam	Questions are reusable. Exams select questions through the `exam_questions` join table and retain a stable scoring basis during active attempts.
Attempt	One student has at most one active or submitted attempt for an exam; start and submission times are server-recorded.
Answer	One stored answer is allowed per attempt and question through a compound uniqueness rule; updates overwrite the prior selection.
Violation	Every event belongs to an attempt and receives `occurred_at` from the server clock, not from the client.
Deletion	Referenced exams, questions, attempts, answers and violations are protected by foreign-key restrictions.

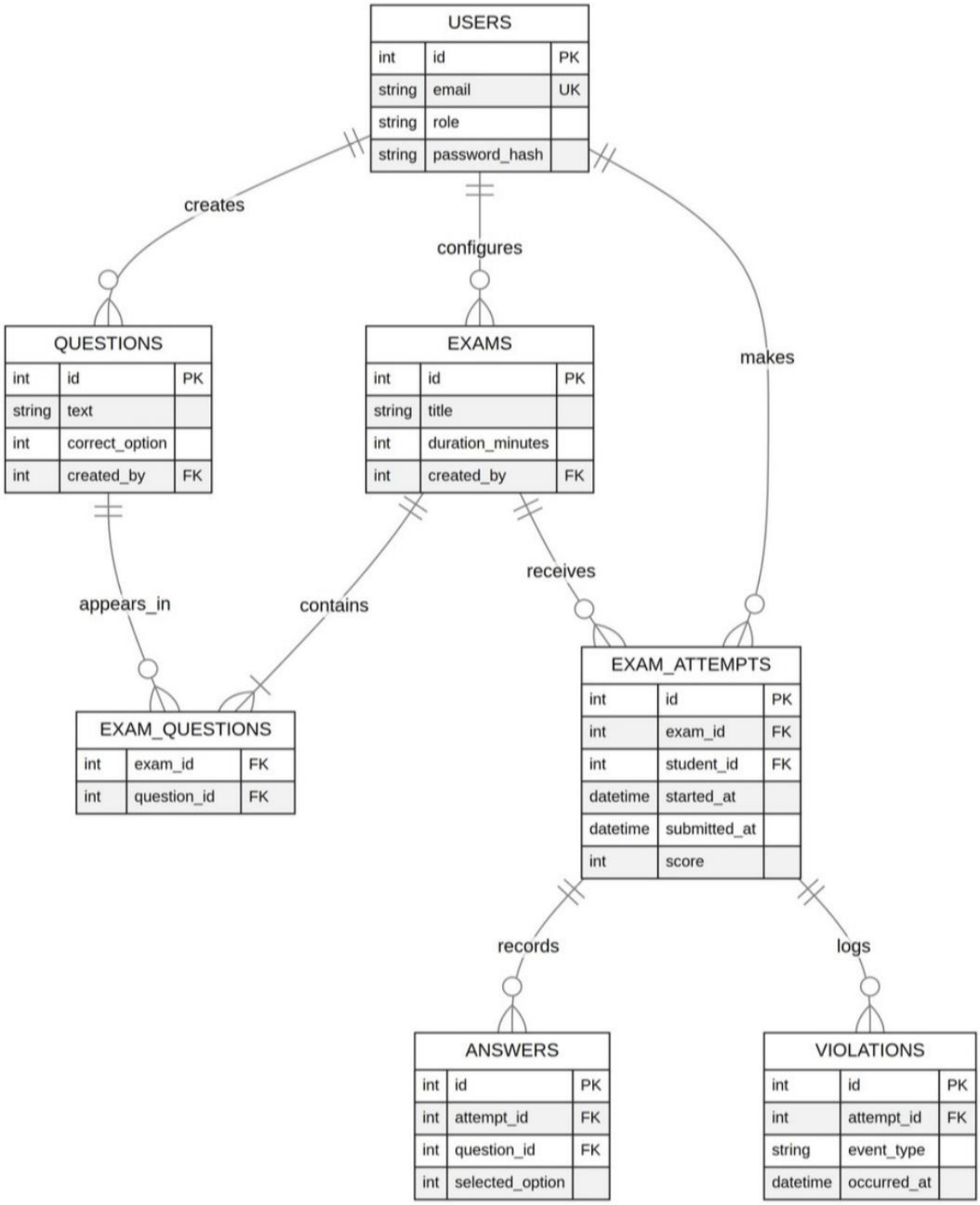


Figure 6.1: Database entities and principal relationships

7. AUTHENTICATION AND AUTHORIZATION ARCHITECTURE

7.1 Authentication and Authorization Overview

Authentication is shared by both roles, but authorization is evaluated on every protected request. A valid login is not sufficient when the requested resource belongs to another student or requires an admin role.

7.2 Control Points

Control Point	Design Behaviour
Credential verification	Load the account by email or username and compare the submitted password with the stored salted hash.
Session creation	Issue a server-recognised session token and return the role needed for dashboard routing.
Role authorization	Reject a valid student token when an admin endpoint is requested; reject missing or invalid tokens with HTTP 401.
Ownership authorization	Ensure a student can access only their own attempt, answer records and result.
Session termination	Invalidate the token on logout and expire idle sessions after the configured timeout.

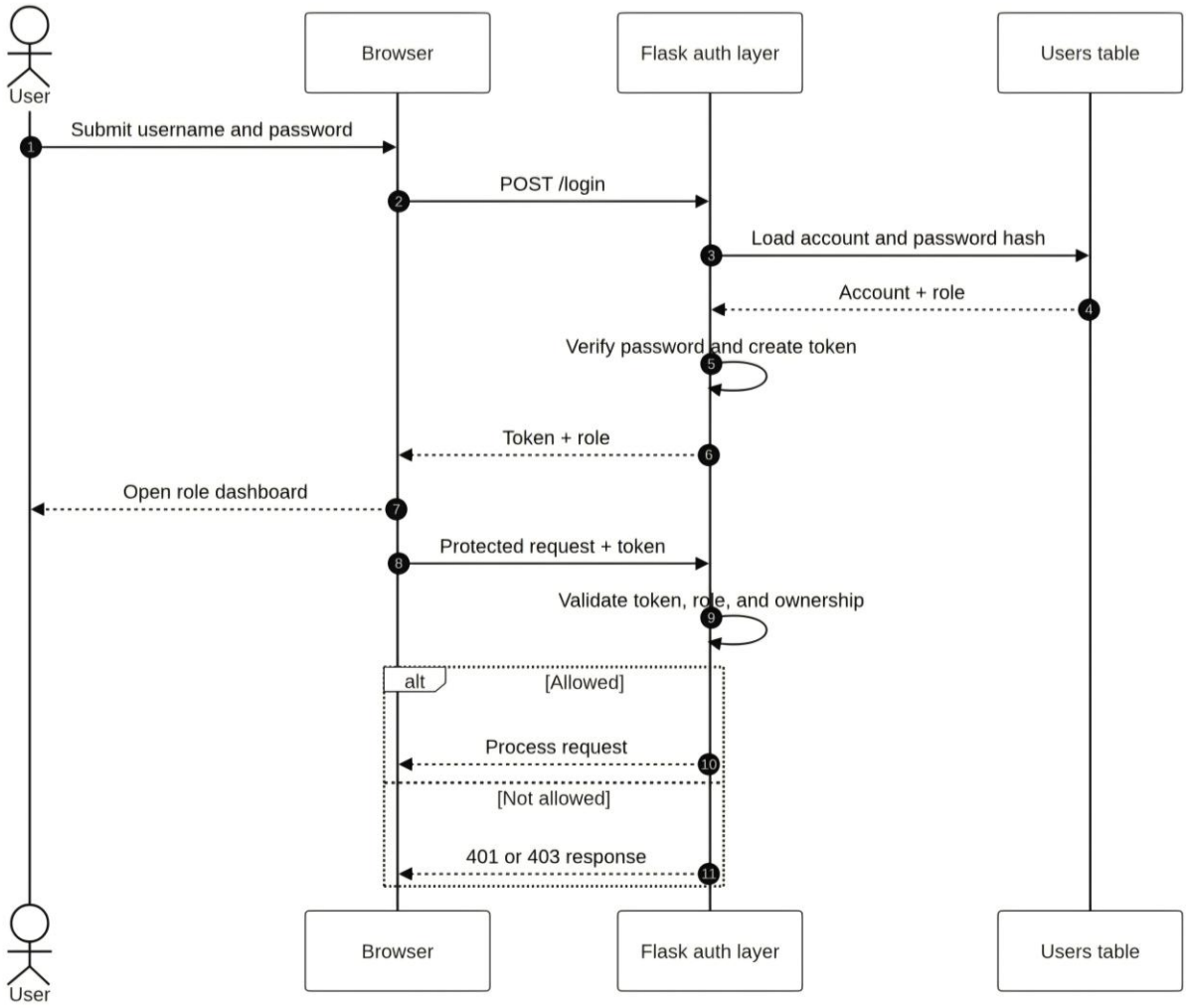


Figure 7.1: Authentication and authorization request sequence

8. DATA FLOW PATTERNS

8.1 Common Request Path

The system uses one consistent request pattern: **authenticate and authorize first, execute a business service second, persist through the repository third, and return a small JSON response last.**

8.2 Main Data Flows

Flow	Main Data Movement
Login	Credentials > auth middleware > users table > token and role response.
Exam attempt	Exam selection > attempt creation > question set and deadline > answer upserts > submission.
Timer expiry	Client countdown reaches zero > submission request > server deadline check > auto-submitted attempt.
Proctoring	`visibilitychange` or Beacon > violation endpoint > active-attempt check > server timestamp > violation row.
Reporting	Submitted attempt > answer-key comparison > score > student result and admin report projection.

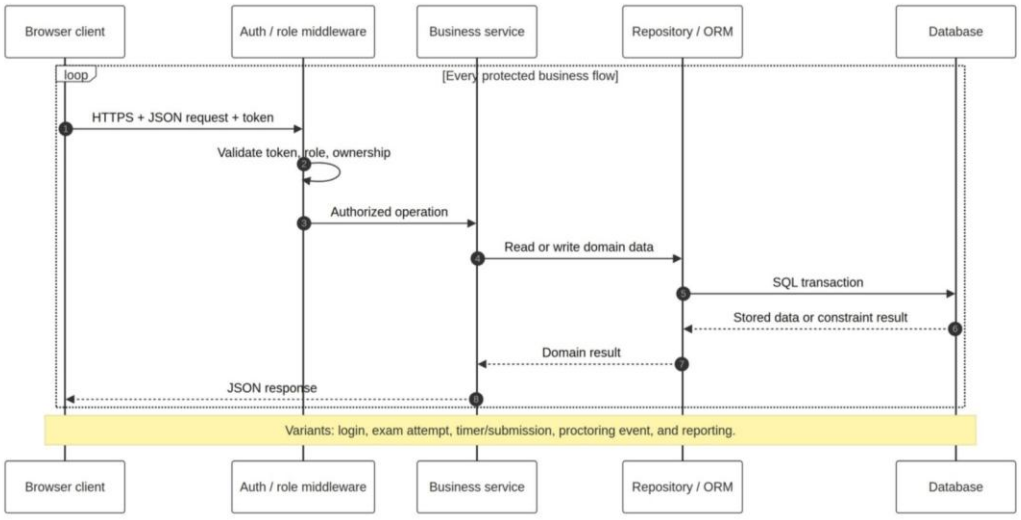


Figure 8.1: Common data-flow path and business-flow types

9. SECURITY ARCHITECTURE

9.1 Security Overview

Security is implemented as a set of small controls placed at the request boundary, the service boundary and the persistence boundary. The academic build is **not presented as a public-internet hardened system**.

9.2 Security Controls

Security Control	Threat or Failure Reduced
HTTPS outside loopback	Reduces interception of credentials, session tokens and exam requests in non-local deployment.
Salted password hashing	Prevents plaintext password persistence and reduces damage from a database disclosure.
Token, role and ownership checks	Prevents unauthenticated access, privilege escalation and cross-student data access.
Server-side validation and ORM binding	Reduces malformed input, SQL injection and invalid state transitions.
Answer-key protection	The correct option is never sent to the browser while an attempt is in progress.
Server-side time and timestamps	Prevents client clock drift from extending exams or falsifying violation history.
Scope limitation	No webcam, audio, screen recording or claim of complete malpractice prevention.

9.3 Security Responsibilities

User Type	Security Responsibility
Student	Keep credentials private; use one attempt per exam; remain on the exam tab during the attempt.
Admin	Protect the admin account; avoid editing questions of an exam with attempts in progress; treat the violation report as a signal for human review.
Developer / deployer	Use HTTPS outside local loopback; supply secret key and database URL through environment variables; review security before any public deployment.

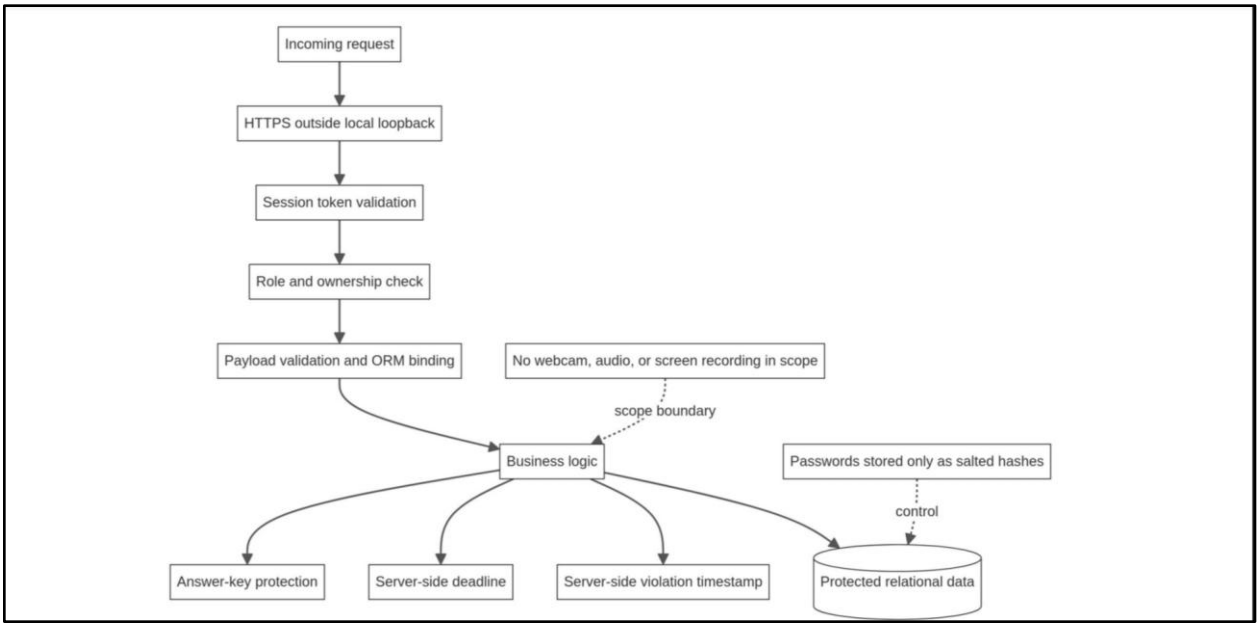


Figure 9.1: Security-control placement

10. DEPLOYMENT ARCHITECTURE

10.1 Deployment Overview

The initial deployment can run as **one backend instance**. A WSGI server is used beyond local development, while SQLite is sufficient for demonstration and MySQL is the production-equivalent option.

10.2 Deployment Components

Node or Artifact	Deployment Role
Student browser	Desktop Chrome, Firefox or Edge session for login, exam, timer and visibility events.
Admin browser	Desktop browser session for question management, configuration, dashboards and reporting.
WSGI server	Receives HTTP requests and runs the Flask application behind a production-style process boundary (for example Gunicorn).
Flask application	Hosts REST endpoints, services, session checks, scoring, proctoring and reporting logic.
Database	SQLite for local development; MySQL for shared or hosted deployment through SQLAlchemy.
Git / GitHub	Stores versioned source, setup instructions, tests and reproducibility artifacts.

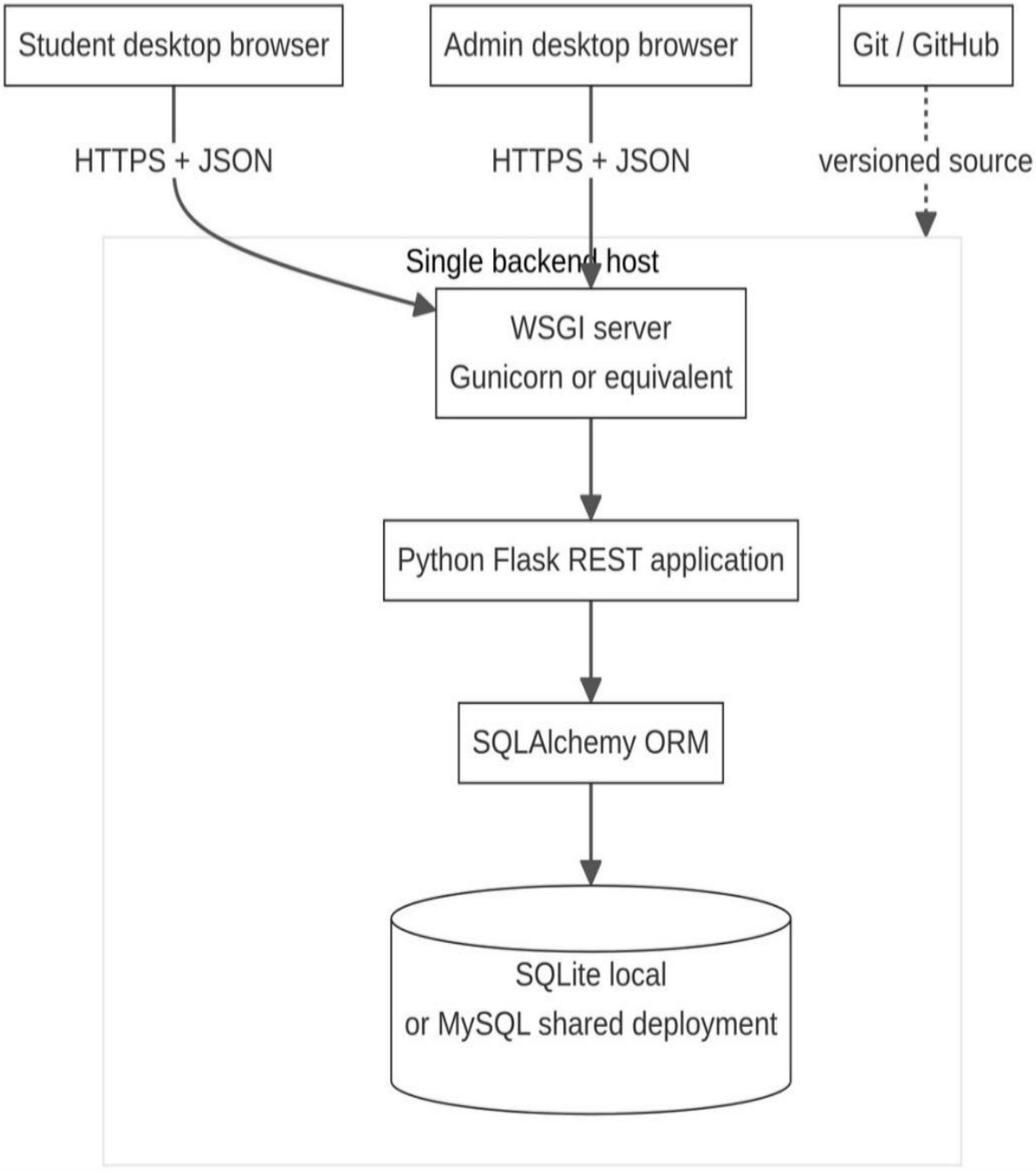


Figure 10.1: Deployment boundary and runtime path

11. KEY DESIGN DECISIONS

The following decisions are architectural constraints rather than optional implementation details. Each decision directly answers a risk or requirement stated in the SRS.

Decision	Reason	Consequence
Server is timing authority	Client clocks can drift or be manipulated.	Every answer and submission request checks the server deadline.
One active attempt	Multiple tabs or devices could create inconsistent records.	The database and start request enforce one attempt per student and exam.
Server-side scoring	The client must not determine its own result.	The answer key remains in the backend and scoring runs after submission.
Server violation timestamp	Client timestamps are not trustworthy.	The receipt time becomes the official `occurred_at` value.
Modular Flask services	Core rules must not be duplicated across routes.	Authentication, exam, proctoring, scoring and reporting remain independently testable.
SQLite-to-MySQL path	The project needs simple local setup and a shared deployment option.	SQLAlchemy isolates most database-specific code.
Lightweight browser proctoring	The academic scope is narrow and camera-free.	The system records a signal for human review rather than claiming full invigilation.

Implementation consequence: tests should concentrate on timer-expiry enforcement, single-attempt enforcement, score calculation, authorization ownership and server-side violation logging.

12. FUNCTIONAL ARCHITECTURE

12.1 Student Functional Architecture

Function	Frontend Component	Backend Component / Data
Login	Login screen	Auth service; users table
View available exams	Student dashboard	Exam service; exams table
Start exam	Exam page	Exam service; exam_attempts table
Answer questions	Exam page; current answer state	Exam service; answers table
Countdown	Timer service	Exam service; server-computed deadline
Tab-switch detection	Proctoring listener (`visibilitychange`, Beacon)	Proctoring service; violations table
Submit (manual or auto)	Exam page; timer expiry	Exam service and scoring service
View own result	Result screen	Scoring / reporting service; exam_attempts table

12.2 Administrator Functional Architecture

Function	Interface	Backend Operation / Data
Login	Login screen	Auth service; role flag
Manage questions	Question Bank Manager	Admin service; questions table
Configure exams	Exam Configuration	Admin service; exams and exam_questions tables
Platform summary	Admin Dashboard	Reporting service; counts of exams and attempts
Review report	Report Viewer	Reporting service; attempts, answers and violations
Export CSV	Report Viewer	Reporting service; CSV generation
Student overview	Admin Dashboard	Reporting service; users and exam_attempts tables

13. NON-FUNCTIONAL ARCHITECTURE

The table below shows how the architecture responds to each quality attribute of the SRS.

Quality Attribute	Design Response
Performance	Target read endpoints below 500 ms and answer/submission endpoints below 1 s under normal lab-scale load.
Timer accuracy	Client display is periodically reconciled with the server-computed deadline.
Reliability	Refreshes and brief network interruptions do not reset the server-derived attempt state.
Usability	Prominent timer, clear validation messages and manual-submit confirmation with the unanswered count.
Scalability	Flask can run behind multiple WSGI workers; indexes support attempt and report queries.
Maintainability	Routes, services, repository code, configuration and tests remain separated.

14. TECHNOLOGY STACK

Interface Item	Selected Technology or Format
Frontend	HTML5, CSS3, vanilla JavaScript ES6+ in an evergreen desktop browser.
Backend	Python 3.10+, Flask REST service and a WSGI server beyond local development.
Persistence	SQLite for development and demonstration; MySQL as the shared deployment option through SQLAlchemy.
Browser interfaces	Page Visibility API for `visibilitychange` / `document.hidden` and Beacon API for final close delivery.
Communication	HTTPS for non-local deployment, JSON payloads and a session token header or secure cookie.
Source control	Git and GitHub with local setup instructions and reproducible project history.

15. REQUIREMENT-TO-ARCHITECTURE TRACEABILITY

The matrix maps the major SRS requirements to the architectural location that implements them. It is intentionally architectural; detailed acceptance tests remain in Chapter 8 of the SRS and in the implementation test plan.

SRS Area	Primary Component	Architecture Responsibility
FR-AUTH-01 to 04	Auth service + users table	Provisioning, login, token issuance, logout, session expiry and role checks.
FR-QB-01 to 04	Admin service + questions / exams	Question validation, question reuse, exam configuration, availability filtering and publication stability.
FR-EXAM-01 to 06	Exam service + attempts / answers	Start validation, one-attempt rule, answer upsert, deadline checks, manual submit and auto-submit.
FR-PROC-01 to 04	Proctoring service + violations	Visibility listener, Beacon delivery, active-attempt validation, server timestamp and count concealment.
FR-RES-01 to 04	Scoring / reporting services	Score calculation, student result, admin consolidated report and CSV export.
FR-ADM-01 to 02	Admin dashboard	Platform counts, student overview and exam-history review.
NFR security	Middleware + service controls	Hashing, HTTPS, validation, role / ownership checks, answer-key protection and timestamp integrity.
NFR reliability	Exam service + database	Server-derived deadline on each request and persistence of attempts and logs.

16. FUTURE ARCHITECTURAL ENHANCEMENTS

The items below are **proposed directions only**. They are not part of the specified build and are not described as implemented.

Future Feature	Possible Component	Purpose
Webcam-based face detection	OpenCV-based detection module added to the client / server pipeline	Flag when the student's face is not visible during the attempt.
Randomised question order; difficulty-tiered banks	Extension of the exam service and question bank	Reduce answer sharing; support tiered assessment.
PDF export of the proctoring report	Additional export function in the reporting service	Provide PDF output in addition to CSV.
Multiple-camera or audio anomaly detection	Additional proctoring signals	Stronger AI-assisted proctoring.

17. ARCHITECTURAL SUMMARY

The architecture is intentionally small: a browser client, a Flask service layer and a relational database. The design stays understandable by placing integrity-sensitive rules on the server and keeping every persistent record traceable to a user, exam, attempt or event.

The browser observes and presents; it detects visibility changes and shows the timer, but it never decides the deadline, the score or the violation time. The Flask services validate every request, enforce the single-attempt rule, apply the answer key and write the record. The database preserves that record through unique, check and foreign-key constraints.

18. CONCLUSION

The Architecture Design Document shows that the requirements of the SRS can be satisfied with a three-layer design that is simple enough for a college-level project and strict about the rules that protect exam integrity. Server-authoritative timing, server-side scoring, server-assigned violation timestamps, role and ownership checks, and answer-key protection together make the recorded results dependable. The same document also states the limits of the design: it provides a tab-switch signal for human review, not full invigilation, and it is not a public-internet hardened system.

PROJECT REPORT – SCREENSHOTS, APPENDICES & REFERENCES

Part IV contains the user interface screenshots, the output of the administrator console, supporting attachments, appendices and the reference list. **Every picture in this part is a placeholder.** Each box states which screen of the running system must be captured. Screenshots must come from the real, running application; no screenshot has been generated or described as an actual result in this document.

TABLE OF CONTENTS

Sr. No.	Topic
1	USER INTERFACE SCREENSHOTS – STUDENT
1.1	Login Screen
1.2	Student Dashboard
1.3	Exam Interface
1.4	Manual Submission Confirmation
1.5	Result Screen
2	OUTPUT OF THE SYSTEM – ADMINISTRATOR CONSOLE
2.1	Admin Dashboard
2.2	Question Bank Manager
2.3	Exam Configuration
2.4	Consolidated Report Viewer
2.5	CSV Export Output
2.6	Student Account Overview
3	SUPPORTING ATTACHMENTS
	APPENDICES
	REFERENCES

8. USER INTERFACE SCREENSHOTS – STUDENT

The following screens are used by the Student role. The description under each heading is taken from the interface requirements in Chapter 5 and the functional requirements in Chapter 3, so that the screenshot you insert can be checked against the specification.

8.1 Login Screen

What this screen must show (from the SRS): Username or email and password entry. On success the user is routed to the dashboard that matches the role of the account (FR-AUTH-02). Invalid credentials give a plain-language error (HTTP 401 behaviour).

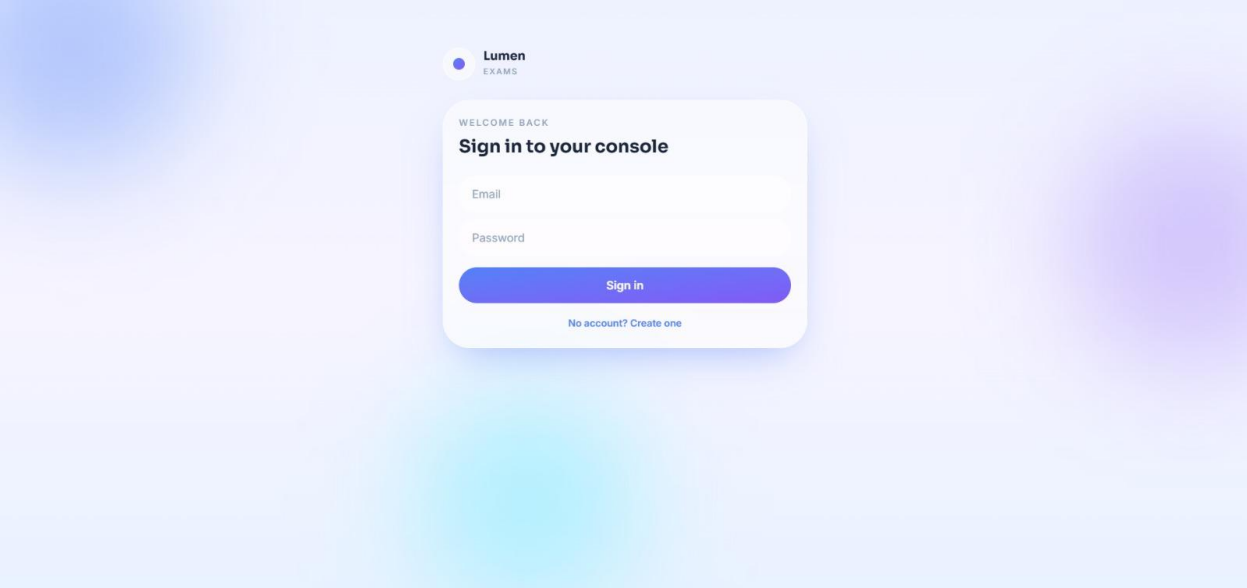


Figure P.1: Login Screen

8.2 Student Dashboard

What this screen must show (from the SRS): The list of exams assigned to the student within their availability window, and the student's past results (FR-QB-04, FR-RES-02).

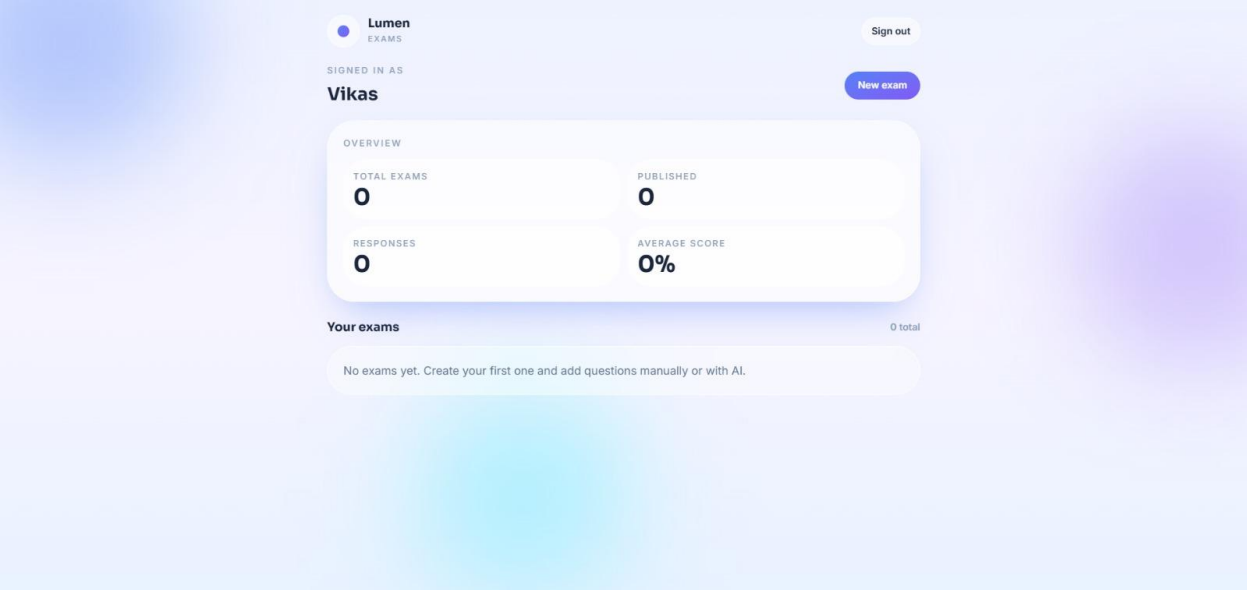


Figure P.2: Student Dashboard

8.3 Exam Interface

What this screen must show (from the SRS): The MCQ presentation with answer selection, the prominently visible countdown timer and the manual submit control. The running violation count must not appear on this screen (FR-PROC-04).

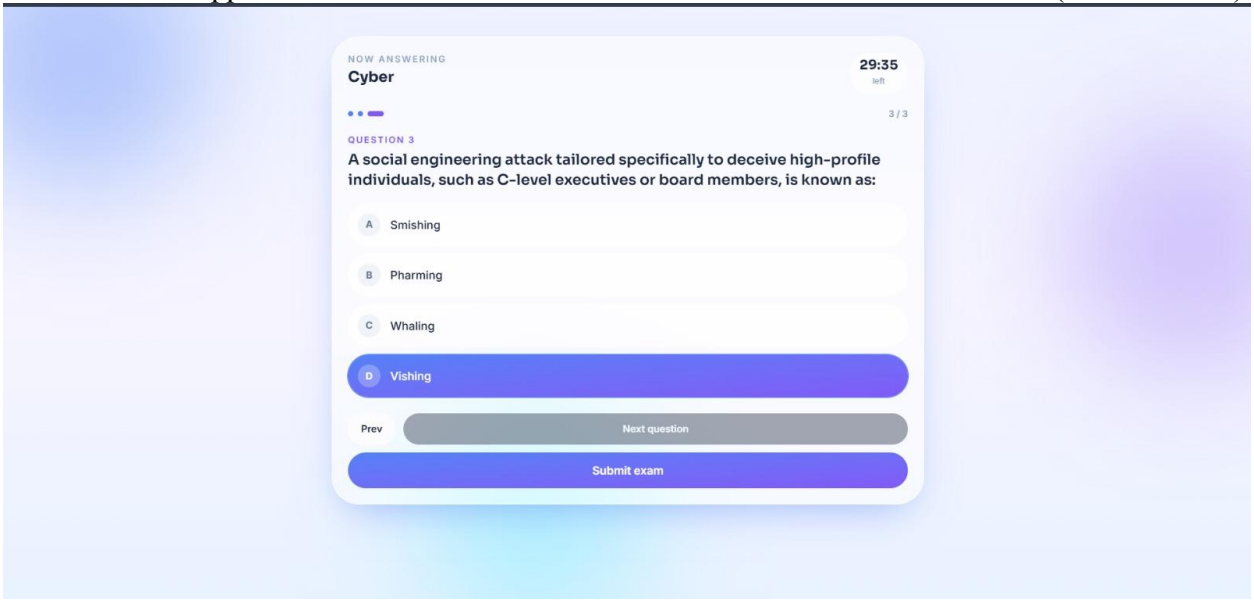


Figure P.3: Exam Interface

8.4 Manual Submission Confirmation

What this screen must show (from the SRS): A confirmation prompt shown before manual submission, stating how many questions are unanswered, if any (Usability requirement, Section 4.5).

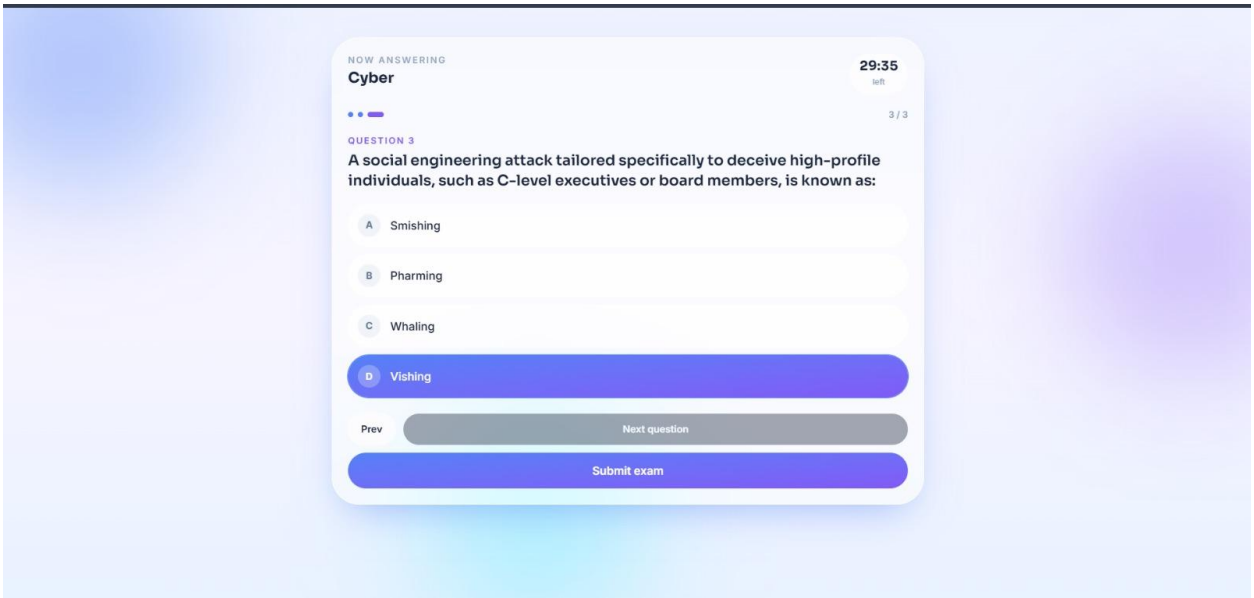


Figure P.4: Manual Submission Confirmation

8.5 Result Screen

What this screen must show (from the SRS): The student's own score, total number of questions and submission time, and no violation log or other students' results (FR-RES-02).

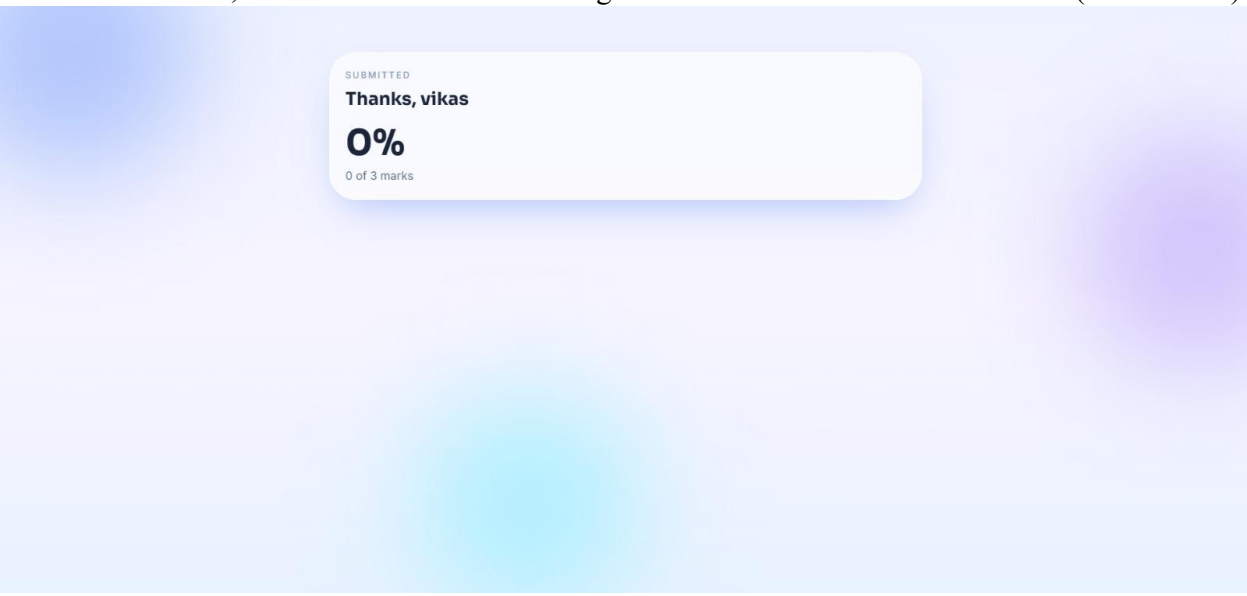


Figure P.5: Result Screen

9. OUTPUT OF THE SYSTEM – ADMINISTRATOR CONSOLE

The following screens are used by the Admin role. As with the student screens, the descriptions come from the specification and are meant to help you verify the screenshots you insert.

9.1 Admin Dashboard

What this screen must show (from the SRS): A summary of platform activity: the number of exams configured, the number of attempts in progress and the number of submitted attempts awaiting review (FR-ADM-01).

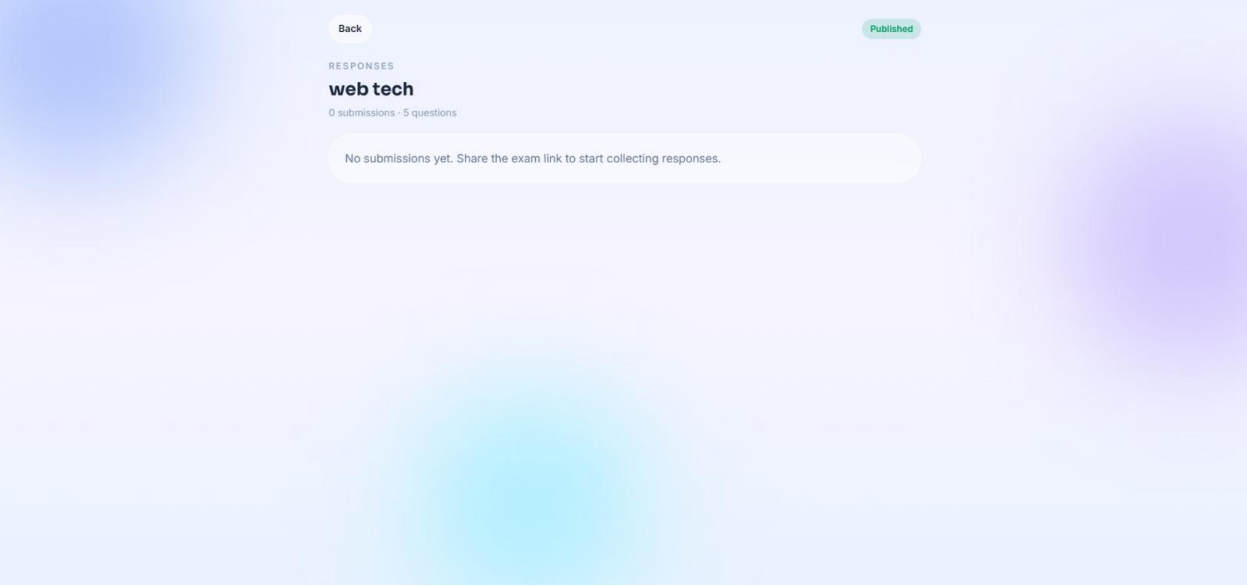


Figure P.6: Admin Dashboard

On-the Job Training (OJT) in SDCA Infotech:



SDAC
Making Nation Employable

CERTIFICATE OF COMPLETION

This is to certify that

Vikas Pankaj Chaurasiya

has Successfully Completed the Internship in

Hybrid Application Development by Using Generative AI

with wonderful remarks at **SDAC INFOTECH**

from **16th Nov 2025 to 05th May 2026**

Dr. Sumathi Rajkumar
Principal
Thakur Ramnarayan College of Arts and Commerce (TRCAC)

MR. Sanjeev Gupta
Technical Head
SDAC Infotech

Successfully completed an On-the-Job Training (OJT) Program in Hybrid Application Development using Generative AI, conducted by SDAC Infotech. The training provided me with an opportunity to explore the field of hybrid application development and understand the role of Generative AI in developing innovative and efficient software solutions.

During this training program, I gained exposure to the fundamentals of hybrid application development and explored the applications of Generative AI in the software development process. The experience helped me understand how AI-powered tools can be utilized to simplify development tasks, improve productivity, and enhance application functionality. It also introduced me to the role of emerging technologies in modern software development and their growing importance in the IT industry.

The program enhanced my awareness of hybrid application development and Generative AI, providing valuable insights into how AI technologies can be integrated into practical applications. It encouraged me to explore technological advancements, improve my technical understanding, and develop an interest in further learning about artificial intelligence and application development.

This training was a valuable step in my academic and professional journey, as it helped me gain practical exposure to industry-oriented learning and understand the potential career opportunities in the technology sector. Overall, completing this program contributed to my learning experience and motivated me to continue developing my technical knowledge, creativity, and problem-solving skills.

Certificate awarded by: SDAC Infotech

Training Domain: Hybrid Application Development using Generative AI

Date of Completion: 5th May 2026

LinkedIn



My LinkedIn profile represents my academic background, professional experience, technical skills, achievements, and career aspirations as a **B.Sc. Computer Science student at Thakur College of Science and Commerce, affiliated with the University of Mumbai**. The profile highlights my professional identity as an **Analytics Engineer and Technical Consultant**, reflecting my interest in data analytics, software development, and emerging technologies.

The profile showcases my technical knowledge and skills in **SQL, Python, Microsoft Excel, Power BI, Data Analytics, and Data Modeling**. These skills reflect my interest in working with data, analyzing information, developing analytical solutions, and understanding how technology supports business decision-making. My LinkedIn profile also highlights my experience as a **Technical Consultant at Skanem India Private Limited**, where I gained exposure to professional responsibilities within the Information Technology sector.

My LinkedIn profile serves as a platform to present my **academic qualifications, internship experiences, technical projects, certifications, professional achievements, and career interests**. It also allows me to connect with industry professionals, recruiters, analytics engineers, software developers, and other individuals working in the technology sector. Through these connections, I aim to expand my professional network, exchange knowledge, and explore opportunities for continuous learning and career development.

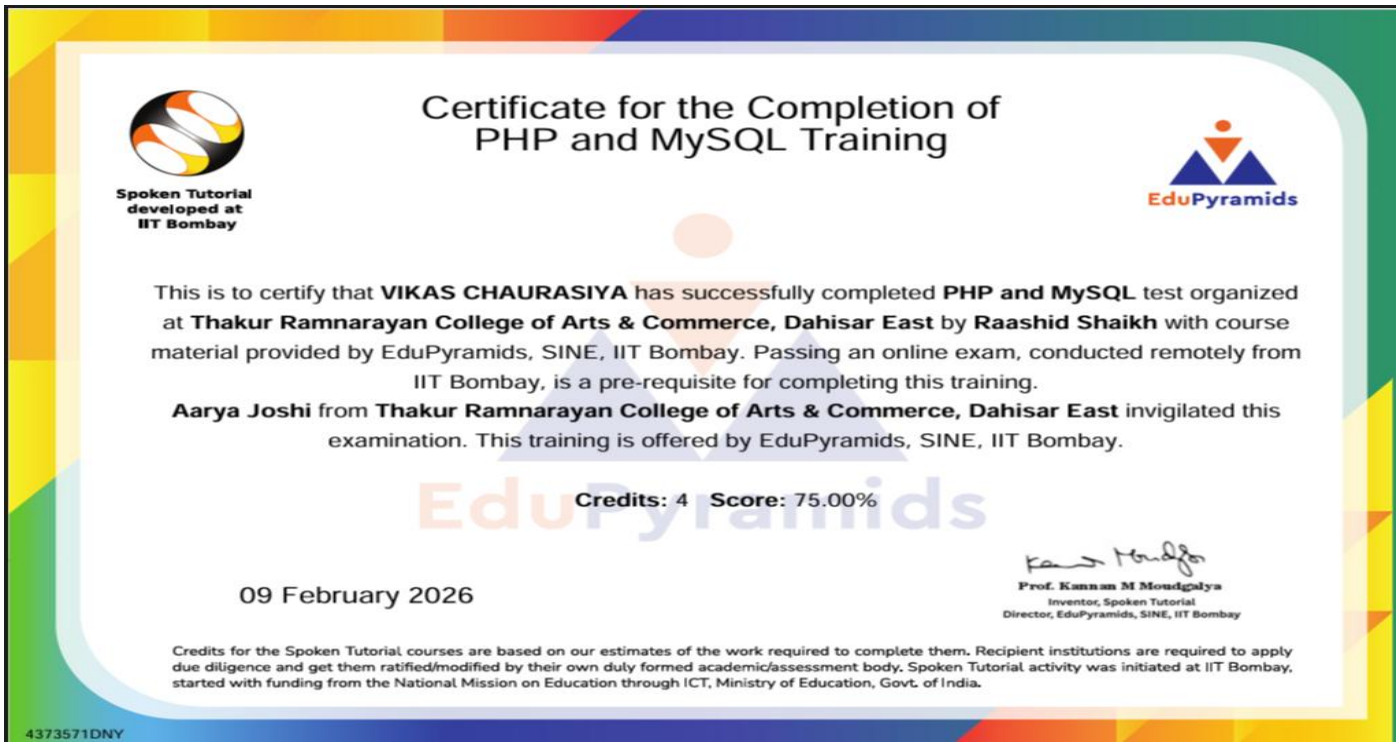
Maintaining an updated LinkedIn profile is an important part of my **academic and professional growth**. It enables me to present my technical capabilities, professional experiences, and accomplishments in an organized and professional manner. The platform also helps me discover career opportunities, stay informed about developments in data analytics and information technology, and build meaningful professional

relationships. Overall, my LinkedIn profile reflects my commitment to developing my technical expertise, improving my analytical and problem-solving skills, and pursuing a successful career in the IT and data analytics industry.

LinkedIn Profile Name: Vikas Chaursariya
Professional Headline: Analytics Engineer | Technical Consultant at Skanem India | SQL • Python • Excel • Power BI | Data Analytics & Data Modeling
Educational Institution: Thakur College of Science and Commerce
Professional Experience: Technical Consultant – Skanem India Private Limited
Area of Interest: Analytics Engineering, Data Analytics, Data Modeling, and Information Technology

Overall, my LinkedIn profile serves as a **digital professional identity**, representing my academic journey, professional experiences, achievements, interests, and continuous efforts toward personal and career growth. It provides a foundation for building a strong professional network and preparing for future career opportunities.

PHP, MySQL and Git Training Certifications



Certificate for the Completion of
PHP and MySQL Training

Spoken Tutorial developed at IIT Bombay

EduPyramids

This is to certify that **VIKAS CHAURASIYA** has successfully completed **PHP and MySQL** test organized at **Thakur Ramnarayan College of Arts & Commerce, Dahisar East** by **Raashid Shaikh** with course material provided by EduPyramids, SINE, IIT Bombay. Passing an online exam, conducted remotely from IIT Bombay, is a pre-requisite for completing this training.
Aarya Joshi from **Thakur Ramnarayan College of Arts & Commerce, Dahisar East** invigilated this examination. This training is offered by EduPyramids, SINE, IIT Bombay.

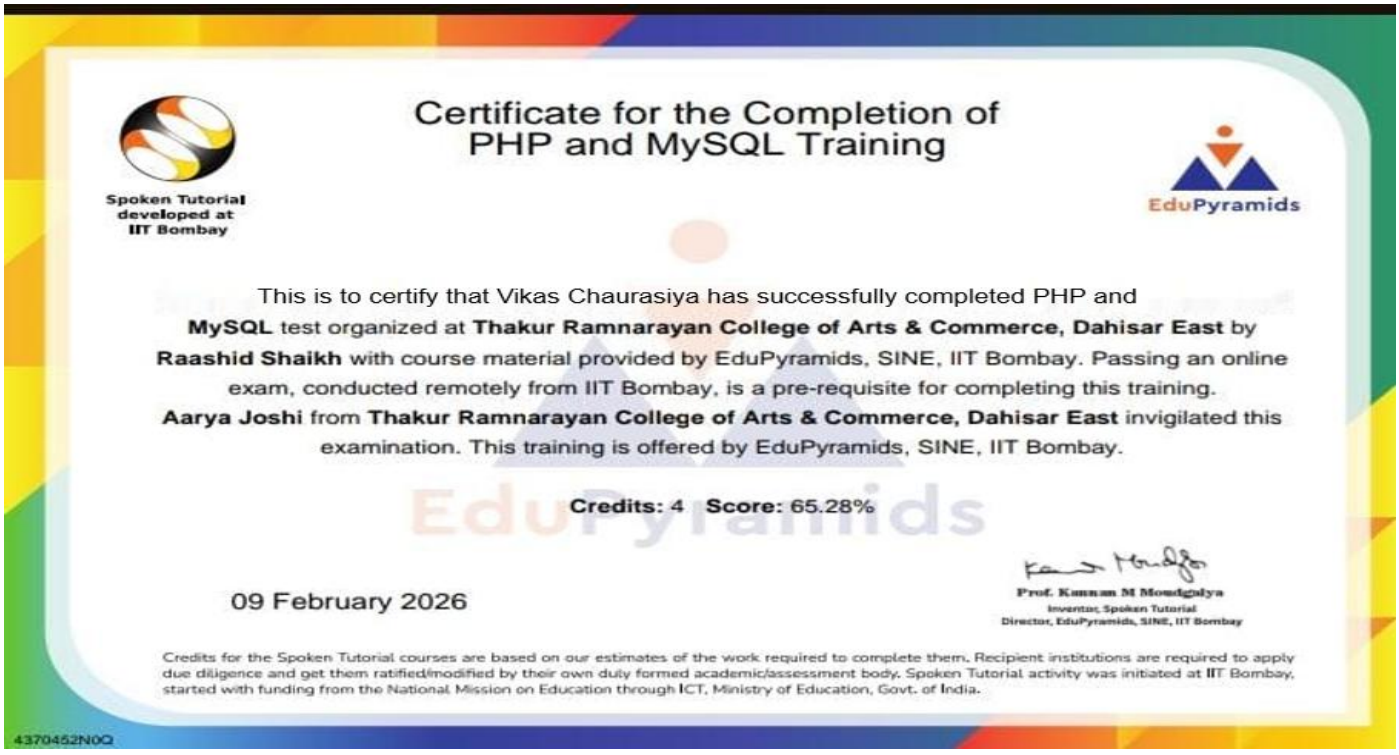
Credits: 4 **Score:** 75.00%

09 February 2026

Prof. Kannan M Moudgalya
Inventor, Spoken Tutorial
Director, EduPyramids, SINE, IIT Bombay

Credits for the Spoken Tutorial courses are based on our estimates of the work required to complete them. Recipient institutions are required to apply due diligence and get them ratified/modified by their own duly formed academic/assessment body. Spoken Tutorial activity was initiated at IIT Bombay, started with funding from the National Mission on Education through ICT, Ministry of Education, Govt. of India.

4373571DNY



Certificate for the Completion of
PHP and MySQL Training

Spoken Tutorial developed at IIT Bombay

EduPyramids

This is to certify that Vikas Chaurasiya has successfully completed PHP and **MySQL** test organized at **Thakur Ramnarayan College of Arts & Commerce, Dahisar East** by **Raashid Shaikh** with course material provided by EduPyramids, SINE, IIT Bombay. Passing an online exam, conducted remotely from IIT Bombay, is a pre-requisite for completing this training.
Aarya Joshi from **Thakur Ramnarayan College of Arts & Commerce, Dahisar East** invigilated this examination. This training is offered by EduPyramids, SINE, IIT Bombay.

Credits: 4 **Score:** 65.28%

09 February 2026

Prof. Kannan M Moudgalya
Inventor, Spoken Tutorial
Director, EduPyramids, SINE, IIT Bombay

Credits for the Spoken Tutorial courses are based on our estimates of the work required to complete them. Recipient institutions are required to apply due diligence and get them ratified/modified by their own duly formed academic/assessment body. Spoken Tutorial activity was initiated at IIT Bombay, started with funding from the National Mission on Education through ICT, Ministry of Education, Govt. of India.

4370452N0Q

Successfully completed **PHP and MySQL Training and Git Training** under the Spoken Tutorial initiative developed by IIT Bombay, with learning resources provided by EduPyramids and SINE, IIT Bombay. These

certifications reflect my commitment to enhancing my technical skills and acquiring additional knowledge beyond my regular academic curriculum in Computer Science.

The **PHP and MySQL Training** was completed on **9 February 2026** at Thakur Ramnarayan College of Arts & Commerce, Dahisar East. As part of the course, I successfully completed the prescribed assessment, which was conducted remotely by IIT Bombay. I secured **65.89% and earned 4 credits**. This training introduced me to the fundamentals of PHP programming and MySQL database management, helping me develop a basic understanding of web development, server-side programming, and database operations.

The **Git Training** was completed on **8 November 2025** through the same Spoken Tutorial initiative. I successfully cleared the assessment with a score of **75.00% and earned 1 credit**. Through this training, I gained knowledge of Git, a widely used version-control system, and learned about its role in tracking code changes, managing project versions, and supporting collaborative software development.

Both certifications strengthened my technical foundation and expanded my understanding of essential tools and technologies used in the software development industry. They provided valuable learning exposure to programming, database management, and version control while encouraging independent learning and practical skill development.

Overall, these training programs reflect my dedication to continuous learning, technical improvement, and professional development. They have helped me broaden my knowledge of computer science concepts and prepare myself for future academic and professional opportunities in the IT field.